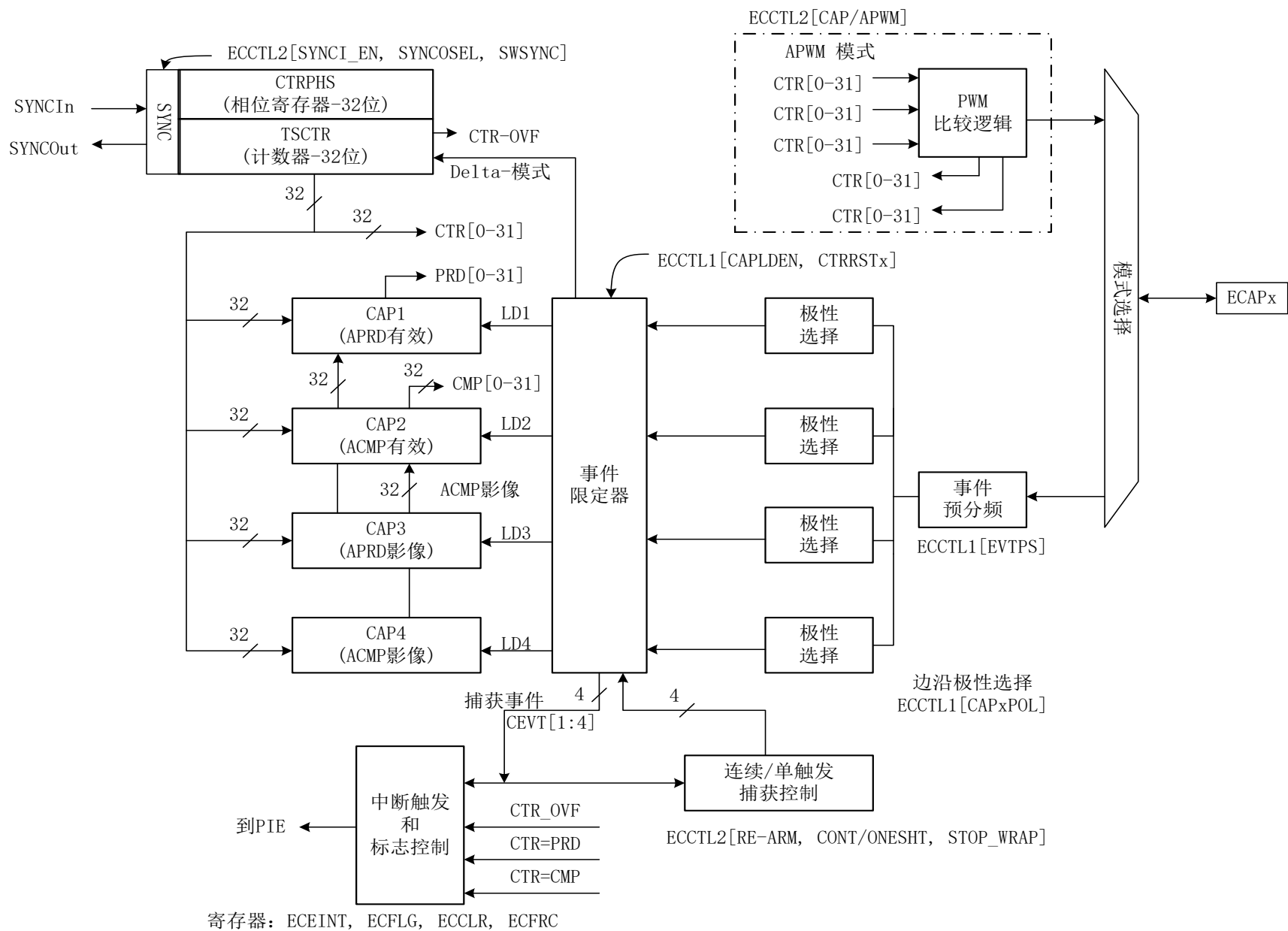


第二部分总结

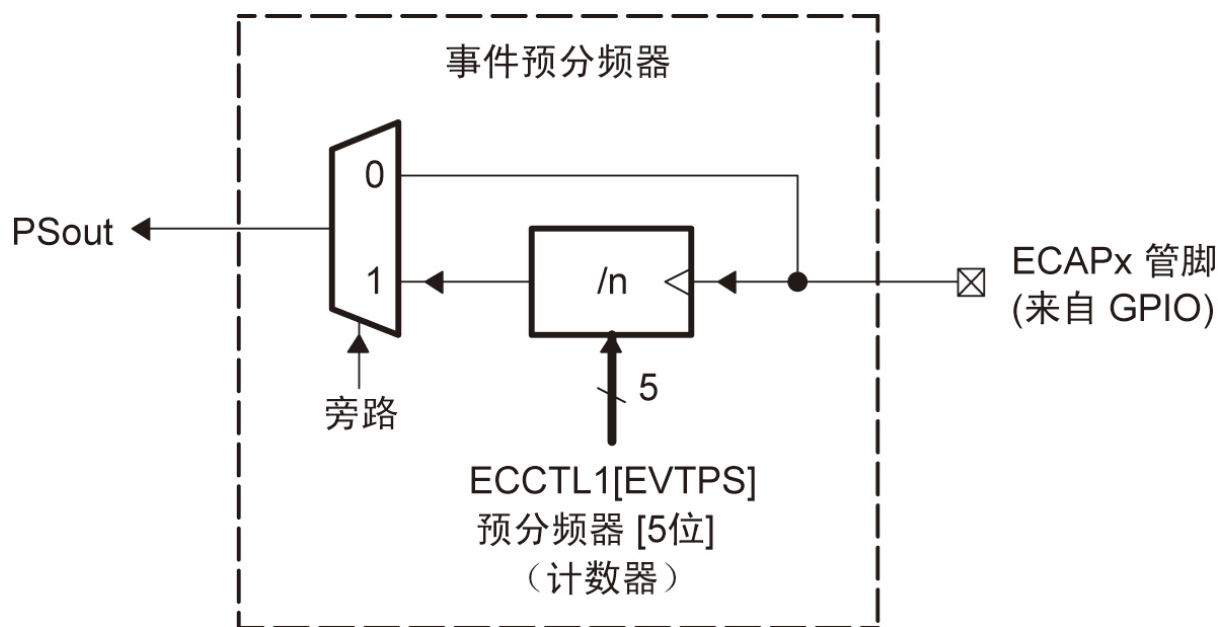
知识点：

1. eCAP — 工作原理、典型应用
初始化、中断
DSP2802x_Ecap. c
2. ePWM — 工作原理、典型应用
初始化、中断
DSP2802x_EPWM. c
3. ADC — 工作原理、典型应用
初始化、中断
DSP2802x_ADC. c



预分频器 (Event-Prescaler)

输入捕获信号（脉冲群）可通过 $N=2-62$ （2的倍数）预分频或可旁路预分频器。当高频信号用作输入时该操作是十分有用的。



A 当选择预分频值为1（即， $ECCTL1[13:9]=0, 0, 0, 0, 0$ ）时，输入捕获信号完全旁路预分频逻辑。

边缘极性选择与限定器

1. 事件捕捉引脚捕捉的脉冲信号经过预分频以后，采用4个独立的边缘极性（上升/下降沿）选择MUX，每个对应一个捕捉事件；
2. 采用mod4序列发生器对每个边沿（最多4个）进行量化；
3. 通过mod4计数器将边沿事件锁存到相应的CAPx寄存器中，CAPx寄存器在下降沿被装载

连续模式：

在连续模式中，

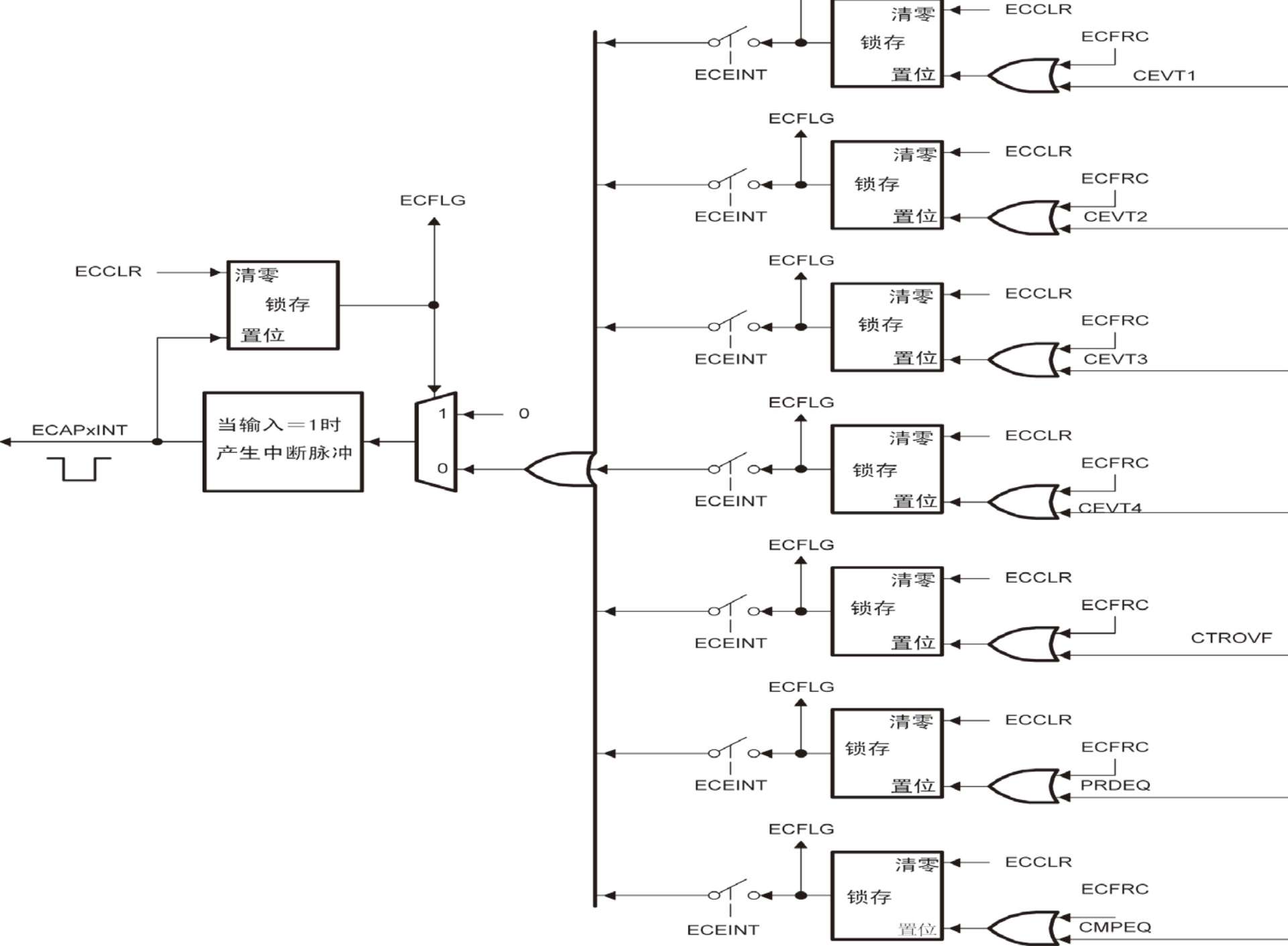
Mod4计数器连续运行（0->1->2->3->0），

单触发操作被忽略，

捕获值在循环缓冲区序列中继续被写入**CAP1-4**。

CAP1-CAP4寄存器

- 这些32位寄存器都连接在32位计数器定时器总线CTR[0-31]上，当选择了相应的装载指令（LD）时进行装载。
- 可以通过使能位CAPLDEN控制寄存器的装载。在单次控制模式中，当mod4计数器满足停止条件时，改位自动清零（禁止装载），即，
StopValue=Mod4



4) eCAP控制寄存器1 (ECCTL1) 字段描述

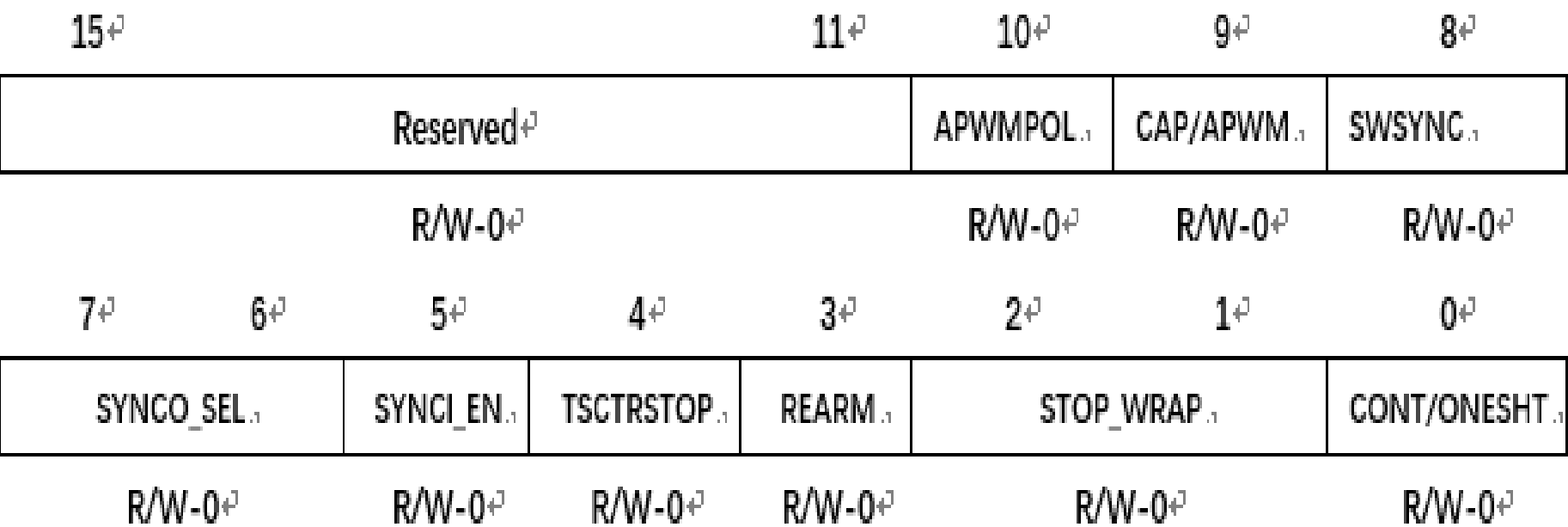
[illegible]

位	字段	值	描述
15:14	FREE/SOFT	00 01 1x	模拟控制 在模拟中止时TSCTR计数器立即停止 TSCTR计数器运行直至=0 TSCTR计数器不受模拟中止的影响（自由运行）
13:9	PRESCALE	00000 00001 00010 00011 00100 00101 ... 11110 11111	事件过滤器预分频选择 通过1分频（即，无预分频、旁路预分频器） 通过2分频 通过4分频 通过6分频 通过8分频 通过10分频 通过60分频 通过62分频
8	CAPLDEN	0 1	在出现捕获事件时使能装载CAP1-4寄存器 在捕获事件时禁止装载CAP1-4寄存器 在捕获事件时使能装载CAP1-4寄存器

位	字段	值	描述
7	CTRRST4	0 1	在出现捕获事件4时计数器复位 在捕获事件4出现时不复位计数器（绝对时间戳操作） 在捕获事件4时间戳已被捕获后复位计数器（用于差分模式操作）
6	CAP4POL	0 1	捕获事件4极性选择 在上升沿（RE）上触发捕获事件4 在下降沿（FE）上触发捕获事件4
5	CTRRST3	0 1	在出现捕获事件3时计数器复位 在捕获事件3出现时不复位计数器（绝对时间戳操作） 在事件3时间戳已被捕获后复位计数器（用于差分模式操作）
4	CAP3POL	0 1	捕获事件3极性选择 在上升沿（RE）上触发捕获事件3 在下降沿（FE）上触发捕获事件3

位	字段	值	描述
3	CTRRST2	0 1	在出现捕获事件2时计数器复位 在捕获事件2出现时不复位计数器（绝对时间戳操作） 在事件2时间戳已被捕获后复位计数器（用于差分模式操作）
2	CAP2POL	0 1	捕获事件2极性选择 在上升沿（RE）上触发捕获事件2 在下降沿（FE）上触发捕获事件2
1	CTRRST1	0 1	在出现捕获事件1时计数器复位 在捕获事件1出现时不复位计数器（绝对时间戳操作） 在事件1时间戳已被捕获后复位计数器（用于差分模式操作）
0	CAP1POL	0 1	捕获事件1极性选择 在上升沿（RE）上触发捕获事件1 在下降沿（FE）上触发捕获事件1

5) ECAP控制寄存器2（ECCTL2）



位	字段		描述
15:11	Reserved		保留
10	APWMPOL	0 1	APWM输出极性选择。这仅可用于APWM操作模式 输出高电平有效（即，比较值定义了高电平时间） 输出低电平有效（即，比较值定义了低电平时间）
9	CAP/APWM	0 1	CAP/APWM操作模式选择 0 ECAP模块在捕获模式下操作。该模式强制下面的配置： 禁止TSCTR通过CTR=PRD事件复位； 禁止在CAP1和CAP2寄存器上装载影像； 允许用户使能CAP1-4寄存器装载； CAPx/APWMx管脚用作捕获输入 1 ECAP模块在APWM模式下操作。该模式强制下面的配置： 在出现CTR=PRD事件时复位TSCTR（周期边界）； 允许在CAP1和CAP2寄存器上实现影像装载； 禁止将时间戳装载到CAP1-4寄存器中； CAPx/APWMx管脚用作APWM输出

位	字段		描述
8	SWSYNC	0 1	软件强制的计数器（TSCTR）同步。这提供了一种便利的软件方法来同步某些或所有ECAP时基。在APWM模式中，同步也可通过CTR=PRD事件来实现写0没有影响。读总是返回0 假设SYNCO_SEL位为0,0，则写1强制当前ECAP模块和任何ECAP模块下行（down-stream）的TSCTR影像装载。在写1后，该位返回0 注：选择CTR=PRD仅在APWM模式中有意义；但是，如果你认为该操作有用，则可在CAP模式中进行选择
7:6	SYNCO_SEL	00 01 10 11	同步输出选择 把同步输入事件选为同步输出信号（直接通过） 把CTR=PRD事件选为同步输出信号 禁止同步输出信号 禁止同步输出信号
5	SYNCI_EN	0 1	计数器（TSCTR）同步输入选择模式 禁止同步输入选项 在出现SYNCI信号或S/W强制事件时使能计数器（TSCTR）从CTRPHS寄存器中装载

4	TSCTRSTOP	0 1	时间戳（TSCTR）计数器停止（冻结）控制 TSCTR停止 TSCTR自由运行
3	RE-ARM	0 1	单触发重新产生（re-arming）控制，也就是等待停止触发。注：重新产生功能在单触发模式或连续模式中均有效 没有影响（读总是返回0） 产生单触发的顺序如下： 1）将Mod4计数器复位为0 2）恢复Mod4计数器 3）使能捕获寄存器装载

2:1	STOP_WRAP	00	单触发模式的停止值。这是在CAP（1-4）寄存器冻结（即捕获序列停止）之前允许出现的捕获数（1-4） 连续模式的循环值。这是在循环缓冲区中不断循环的捕获寄存器的数（在1至4间） 在单触发模式中捕获事件1后停止 在连续模式中捕获事件1后循环
		01	在单触发模式中捕获事件2后停止 在连续模式中捕获事件2后循环
		10	在单触发模式中捕获事件3后停止 在连续模式中捕获事件3后循环
		11	在单触发模式中捕获事件4后停止 在连续模式中捕获事件4后循环 注：STOP_WRAP与Mod4计数器相比较，当相等时，出现2种操作： Mod4计数器停止（冻结）； 禁止捕获寄存器装载 在单触发模式中，阻止更多的中断事件直至重新产生中断事件
0	CONT/ONESHT	0	连续或单触发模式控制（仅在捕获模式中可以应用） 在连续模式中操作
		1	在单触发模式中操作

6) 中断相关寄存器

包括中断使能寄存器 (ECEINT) 、
中断标志寄存器 (ECFLAG) 、
中断清零寄存器 (ECCLR)
和中断强制寄存器 (ECFRC) 。

(1) ECAP 中断使能寄存器 (ECEINT) ⁴

Reserved ⁴							
15 ⁴							8 ⁴
7 ⁴	6 ⁴	5 ⁴	4 ⁴	3 ⁴	2 ⁴	1 ⁴	0 ⁴
CTR=CMP ⁴	CTR=PRD ⁴	CTROVF ⁴	CEVT4 ⁴	CEVT3 ⁴	CEVT2 ⁴	CEVT1 ⁴	Reserved ⁴
		R/W ⁴	R/W ⁴	R/W ⁴	R/W ⁴	R/W ⁴	

(2) ECAP 中断标志寄存器 (ECFLAG) ↵

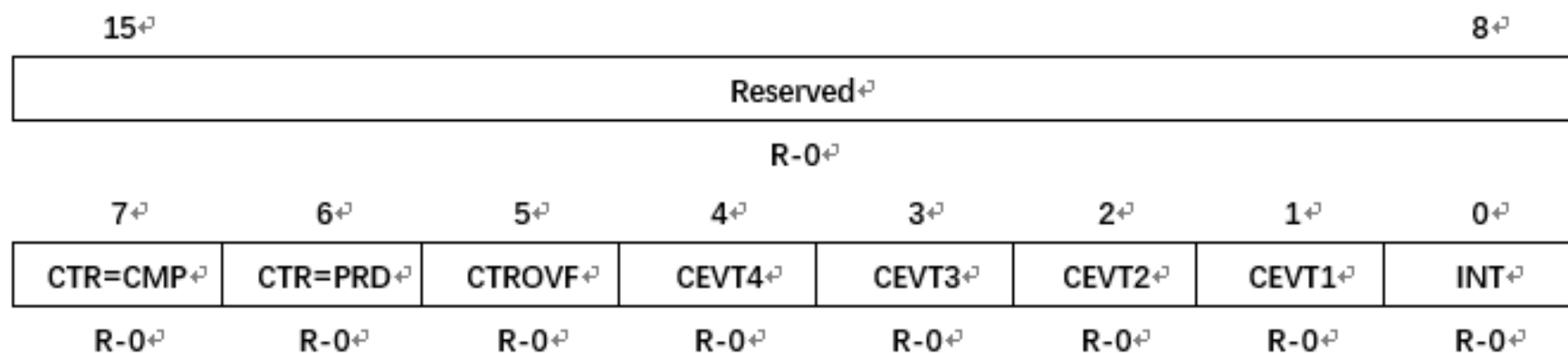


图 8.7 ECAP 中断标志寄存器 (ECFLAG) ↵

eCAP1应用举例:

小车转速测量：智能小车车轮周长20CM，车轮每转1圈测速传感器输出10个脉冲。

实现思路：测速传感器的信号脉冲接入到CAP1，脉冲的频率折算成转速，需要考虑高转速和低转速的测量需求。

要求：完成CAP1的初始化，中断服务程序名为Ecap1Int_isr;

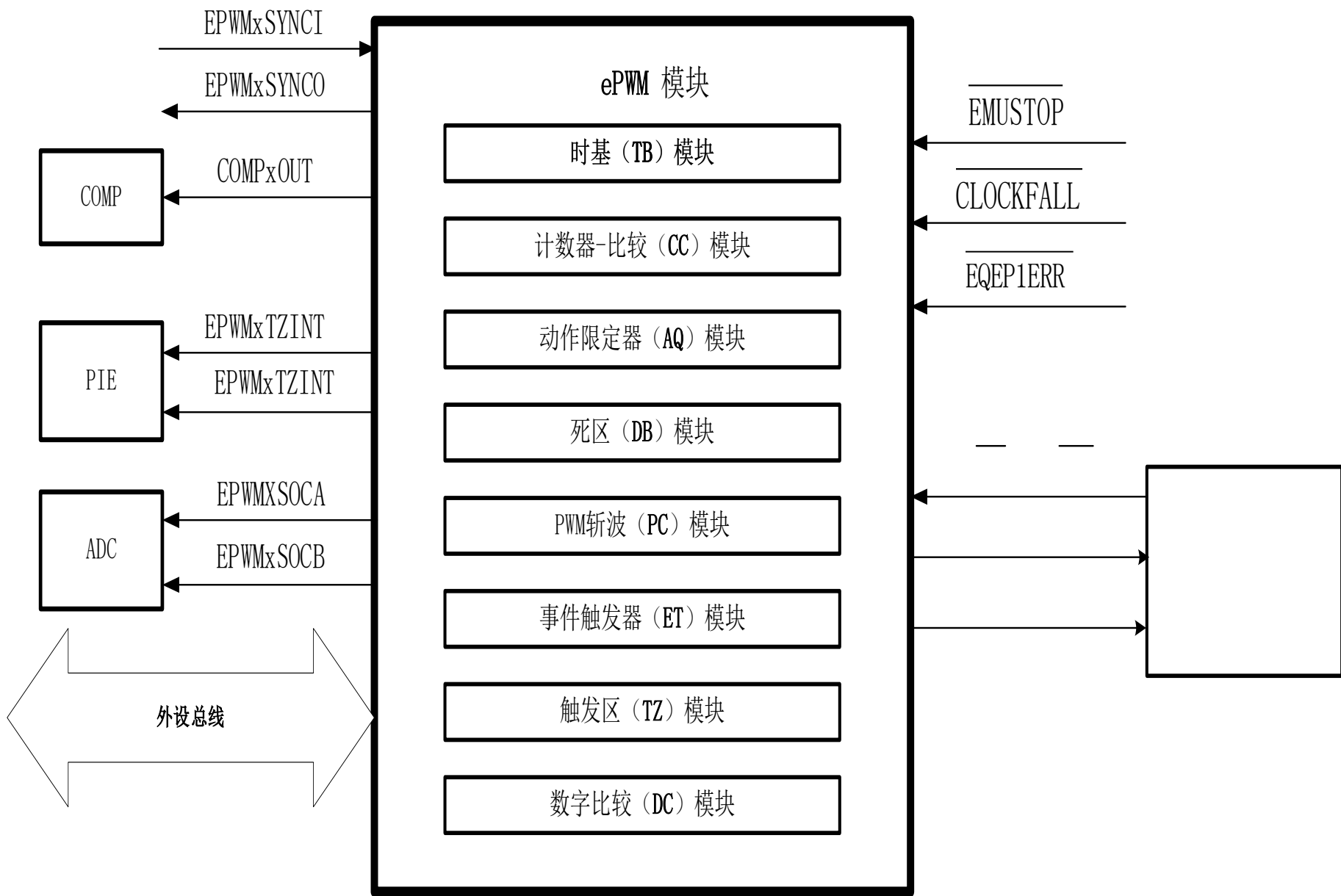
```
extern interrupt void Ecap1Int_isr();
```

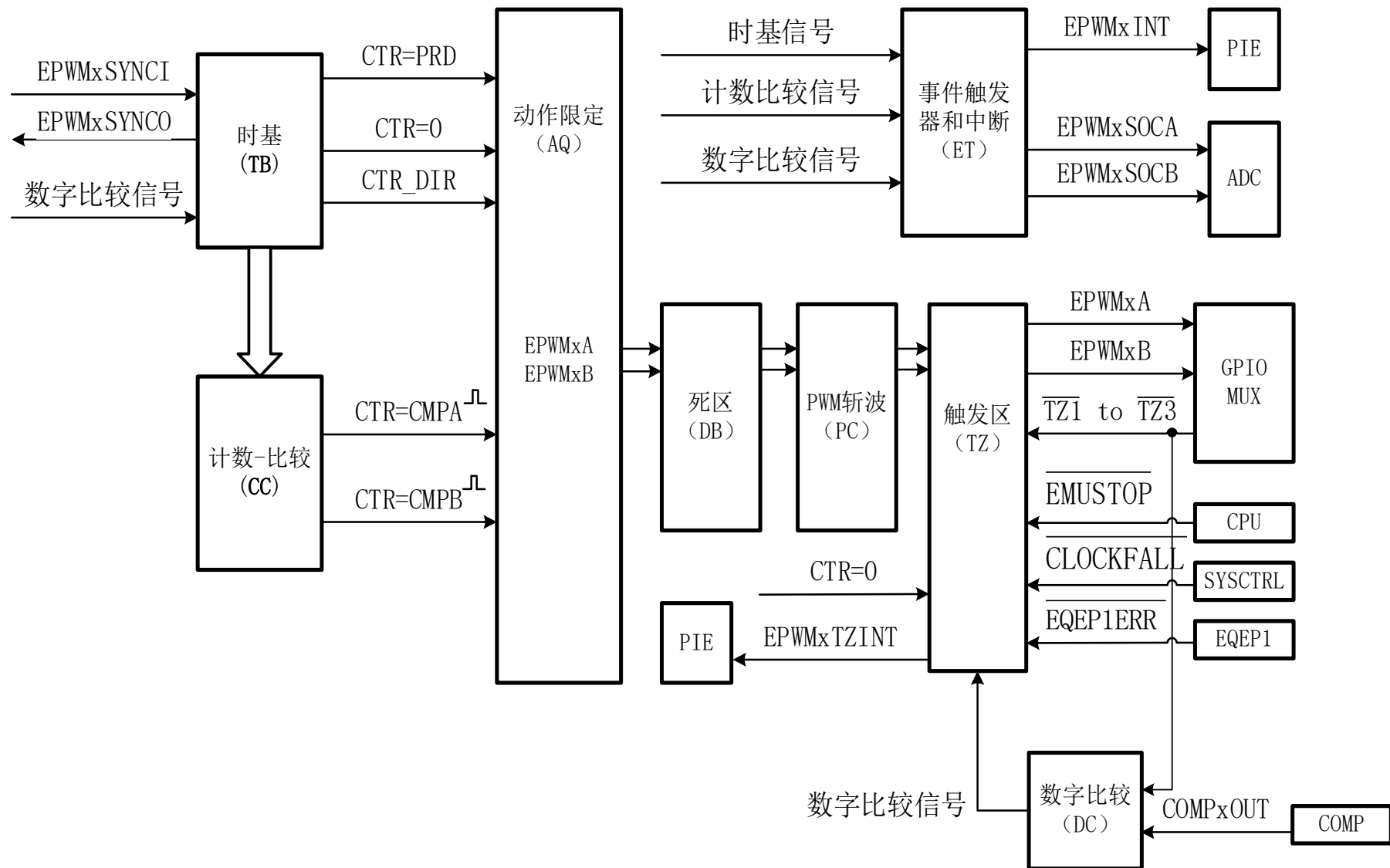
```
void InitCap1()
```

```
{  
    _____(1)_____;EALLOW  
    GpioCtrlRegs.GPAMUX1.bit.GPIO5 = _____(2)_____; 3  
    ECap1Regs.ECCTL1.all = _____(3)_____; 0x01aa/0x41aa/0x81aa  
    // 捕获每个正跳沿，差分时间戳  
    ECap1Regs.ECCTL2.all = _____(4)_____; 0x0010;  
    ECap1Regs.ECEINT.all = _____(5)_____; 0x30 //  
    CEVT4, CTROVF中断允许  
    PieVectTable.ECAP1_INT = _____(6)_____;@Ecap1Int_isr  
    PieCtrlRegs.PIEIER4.bit._____(7)_____ = 1;INTx1 //ECAP1的PIEIER使能  
    IER |= _____(8)_____;0x8 or M_INT4  
    _____(9)_____;EDIS  
}
```

```
interrupt void Ecap1Int_isr(void)
{
    li1=ECap1Regs.CAP1;
    li2=ECap1Regs.CAP2;
    li3=ECap1Regs.CAP3;
    li4=ECap1Regs.CAP4;
    PWM_HI=((li2-li1)+(li4-li3)) >> 1;
    PWM_LO=li3-li2;
    PWM_PRD=((li3-li1)+(li4-li2)) >> 1;

    ECap1Regs.ECCLR.bit.CEVT4 = 1;
    ECap1Regs.ECCLR.bit.INT = 1;
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP4;
}
```





八个子模块作用和关系：

1. **TB**: PWM计数器方式、两个事件（最大点、过零点）和计数方向
2. **CC**: 产生两套比较事件，最多四个
3. **AQ**: 对TB和CC所产生事件的动作（置1,清0，翻转，空动作）
4. **DB**: 互补PWM的死区，AQ的生效方式
6. **ET**: AQ的附加作用，触发ADC和中断

- 配置时基时钟的速率；CPU系统时钟（SYSCLKOUT）经过预分频得到的值。时基计数器从而能够以较慢的速率递增/递减
- 指定ePWM时基计数器（TBCTR）的频率或周期，以控制事件发生的频率
- 管理与其他ePWM模块之间的时基同步，维持与其他ePWM模块之间的相位关系
- 时基计数器三种计数模式：递增、递减或者“先递增后递减”
- 产生以下事件：
CTR = PRD（TBCTR = TBPRD）和CTR = 0（TBCTR = 0x0000）

简而言之：

- TB模块决定PWM的时钟、
- 计数器工作模式、
- 计数器周期（PWM周期），
- 产生计数值等于PWM周期和计数值等于0两个事件，
- 并实现PWM单元间的同步。

时基计数器三种工作模式（TBCTL选择）：

先递增后递减模式 (Up-Down mode)：

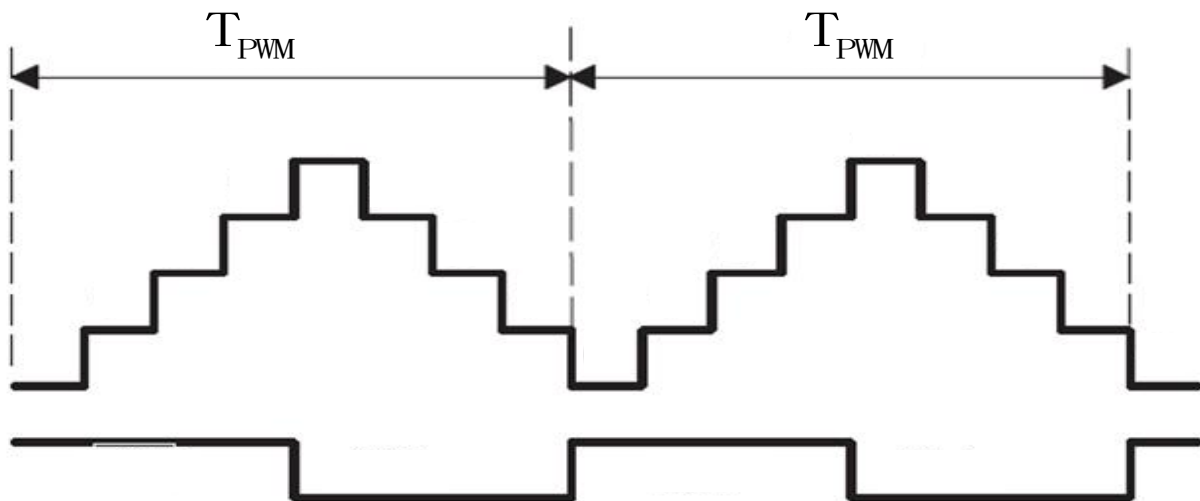
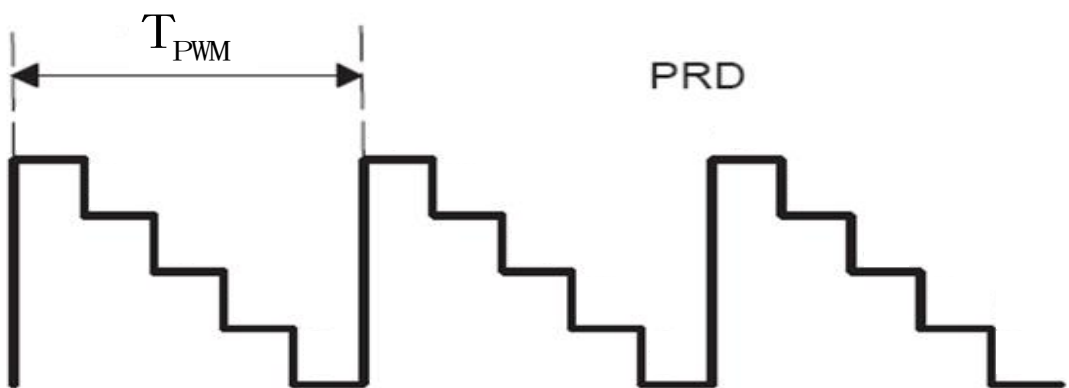
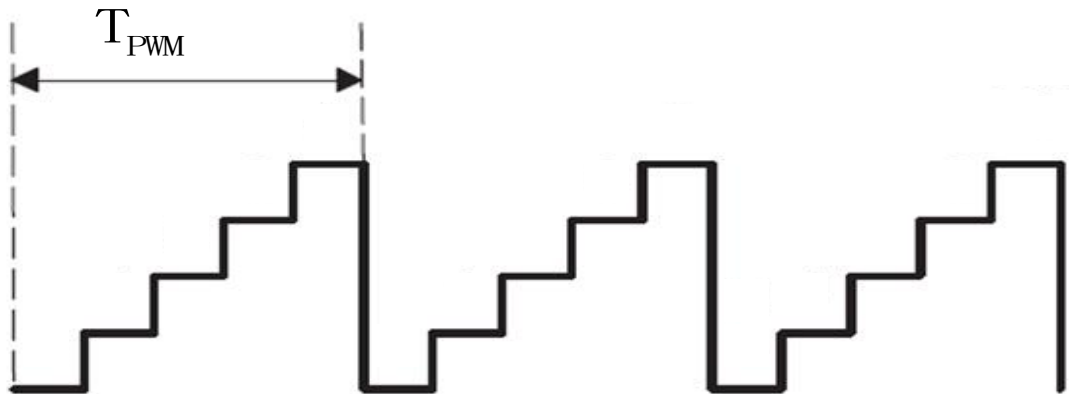
时基计数器从零开始递增计数直到到达周期值（TBPRD），然后在达到周期值时，时基计数器又开始递减直到为0，然后计数器重复这种模式又开始递增。

递增模式 (Up mode)：

时基计数器从0开始递增计数直到到达周期寄存器（TBPRD）中的值。当达到周期值时，时基计数器复位到0并再次开始递增计数。

递减模式 (Down mode)：

时基计数器从周期（TBPRD）值开始递减计数直到为0。当它到达0时，时基计数器复位到周期值并再次开始递减计数。



递增或递减计数模式：

$$T_{\text{PWM}} = (\text{TBPRD} + 1) \times T_{\text{TBCLK}}$$

$$F_{\text{PWM}} = 1 / (T_{\text{PWM}})$$

递增或递减计数模式：

$$T_{\text{PWM}} = 2 \times \text{TBPRD} \times T_{\text{TBCLK}}$$

$$F_{\text{PWM}} = 1 / (T_{\text{PWM}})$$

时基周期寄存器（TBPRD）包含一个映射寄存器。映射寄存器使得寄存器的更新与硬件同步。

定义用于描述ePWM模块中的**所有**映射寄存器:

- **有效（active）寄存器**

有效寄存器控制硬件，并且负责由硬件导致或引发的动作

- **映射（shadow）寄存器**

映射寄存器缓冲或暂时保存有效寄存器的存储单元,它不直接影响任意控制硬件。在特定的时刻，映射寄存器的内容会被转移到有效寄存器,这样做可以避免因寄存器被软件异步修改而发生的错误或虚假操作。

(4) 时基控制寄存器 (TBCTL) :

15 [↕]	14 [↕]	13 [↕]	12 [↕]	11 [↕]	10 [↕]	9 [↕]	8 [↕]
FREE.SOFT [↕]		PHSDIR [↕]	CLKDIV [↕]			HSPCLKDIV [↕]	
R/W-0 [↕]		R/W-0 [↕]	R/W-0 [↕]			R/W-0.0.1 [↕]	
7 [↕]	6 [↕]	5 [↕]	4 [↕]	3 [↕]	2 [↕]	1 [↕]	0 [↕]
HSPCLKDIV [↕]	SWFSYNC [↕]	SYNCOSEL [↕]		PRDLD [↕]	PHSEN [↕]	CTRMODE [↕]	
R/W-0.0.1 [↕]	R/W-0 [↕]	R/W-0 [↕]		R/W-0 [↕]	R/W-0 [↕]	R/W-11 [↕]	

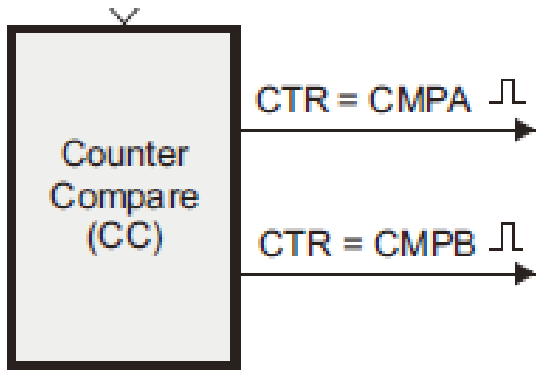
图 9.17 时基控制寄存器 (TBCTL) [↕]

位↵	名称↵	值↵	描述↵
15:14↵	FREE, SOFT↵	↵	仿真模式位。这些位选择 ePWM 时基计数器在仿真期间的行为。↵
13↵	PHSDIR↵	↵ ↵ ↵ ↵ ↵ 0↵ 1↵	相位方向位↵ 这个位仅在时基计数器被配置为“先递增后递减”计数模式时才使用。 PHSDIR 位用于指示在发生同步事件后时基计数器 (TBCTR) 采用何种计数方式计数并从相位 (TBPHS) 寄存器装载新相位值。这与同步事件发生之前计数器的方向无关。在递增和递减计数模式中都不需要这个位。↵ 在同步事件后递减计数↵ 这同步事件后递增计数↵
12:10↵	CLKDIV↵	↵ ↵ 000↵ 001↵ 010↵ 011↵ 100↵ 101↵ 110↵ 111↵	时基时钟预分频位：这些位决定时基时钟的预分频值↵ $TBCLK = SYSCLKOUT / (HSPCLKDIV \times CLKDIV)$ ↵ /1 (复位时的默认值)↵ /2↵ /4↵ /8↵ /16↵ /32↵ /64↵ /128↵

位↵	名称↵	值↵	描述↵
6↵	SWFSYNC↵	0↵ 1↵ ↵	软件强制同步脉冲↵ 写 0 无影响，读取时返回 0↵ 写 1 强制产生一个一次（one-time）同步脉冲↵ 该事件与 ePWM 模块的 EPWMxSYNCCI 输入相或↵ SWFSYNC 仅在 EPWMxSYNCCI 被 SYNCOSSEL = 00 选中时有效（工作）↵
5:4↵	SYNCOSSEL↵	↵ 00↵ 01↵ 10↵ 11↵	同步输出选择。这些位选择 EPWMxSYNCO 信号的源。↵ EPWMxSYNC: ↵ CTR = 0: 时基计数器等于 0 (TBCTR = 0x0000) ↵ CTR = CMPB: 时基计数器等于计数器比较 B (TBCTR = CMPB) ↵ 禁用 EPWMxSYNCO 信号↵
3↵	PRDLD↵	↵ 0↵ 1↵ ↵	有效周期寄存器是否从映射寄存器装载的选择位↵ 当 TBCTR=0 时，周期寄存器 (TBPRD) 从映射寄存器那里装载↵ 立即装载 TBPRD 寄存器，无需使用映射寄存器↵ 对 TBPRD 寄存器执行写或读操作都直接访问有效寄存器。↵
2↵	PHSEN↵	0↵ 1↵	计数器寄存器从相位寄存器装载的使能位↵ 不从时基相位寄存器 (TBPHS) 那里装载时基计数器 (TBCTR) ↵ 当 EPWMxSYNCCI 输入信号出现或者 SWFSYNC 位强制进行软件同步，抑或发生数字比较同步事件时，时基计数器从相位寄存器那里装载。↵
1↵	CTRMODE↵	↵ 00↵ 01↵ 10↵ 11↵	计数模式↵ 递增计数模式↵ 递减计数模式↵ 先递增后递减↵ 停止-停顿模式（复位时的默认值）↵

模块功能:

当计数值到达设定的比较值时，触发相应的事件，如下图所示



$CMPA = TBCTR$

或 $CMPB = TBCTR$ 时

模块会给出事件触发信号给下一个模块（AQ）进行处理。

信号		事件描述	寄存器比较
CTR	=	时基计数器的计数值等于计数器-比较A active寄存器的值	TBCTR = CMPA
CMPA			
CTR	=	时基计数器的计数值等于计数器-比较B active寄存器的值	TBCTR = CMPA
CMPB			
CTR = PRD		时基计数器的计数值等于active周期。 用于从shadow寄存器那里装载active计数器-比较A和B寄存器	TBCTR = TBPRD
CTR	=	时基计数器的计数值等于0。 用于从shadow寄存器那里装载active计数器-比较A和B寄存器	TBCTR = 0x0000
ZERO			

- 计数器-比较子模块负责根据两个比较寄存器产生两个独立的比较事件：
 - CTR = CMPA: 时基计数器等于计数器-比较A寄存器
 - CTR = CMPB: 时基计数器等于计数器-比较B寄存器
- 如果为递增或递减模式，每个事件每个周期仅发生一次。
- 如果为“先递增后递减”模式且比较值在0x0000~TBPRD之间，则每个事件每个周期发生两次，若比较值等于0x0000或等于TBPRD，则每个事件每个周期发生一次。
- 这些事件被馈送到动作限定器子模块，在那里，它们受计数器方向的限制并在使能时被转换成各种动作。

- 计数器-比较寄存器CMPA和CMPB都包含相关的shadow寄存器。shadow寄存器使得寄存器的更新与硬件同步。
- 当使用了shadow寄存器时，active寄存器只在重要时刻进行更新。这样可以避免因寄存器因软件异步修改而发生的讹误或谬误操作。
- active寄存器的存储器地址与shadow寄存器相同。至于哪个寄存器被写或被读，这由CMPCTL[SHDWAMODE]和CMPCTL[SHDWBMODE]位决定。这些位可以分别使能和禁用CMPA shadow寄存器和CMPB shadow寄存器。

shadow模式:

立即装载模式:

(3) 计数器-比较控制寄存器 (CMPCTL) :

15				10		9	8
Reserved						SHDWBFULL	SHDWAFULL
R-0				R-0		R-0	
7	6	5	4	3	2	1	0
Reserved	SHDWBMODE	Reserved	SHDWAMODE	LOADBMODE		LOADAMODE	
R-0	R/W-0	R-0	R/W-0	R/W-0		R/W-0	

位↵	名称↵	值↵	描述↵
15-10↵	Reserved↵	↵	保留↵
9↵	SHDWBFULL↵	↵ ↵ 0↵ 1↵	计数器-比较 B（CMPB）映射寄存器满状态标志↵ 一旦发生装载选通，这个位就会自己清零。↵ 0↵ CMPB 映射 FIFO 还未满↵ 1↵ 表示 CMPB 映射 FIFO 满了；CPU 写操作会覆盖当前映射值↵
8↵	SHDWAFULL↵	↵ ↵ ↵ ↵ ↵ 0↵ 1↵	计数器-比较 A（CMPA）映射寄存器满状态标志↵ 该标志在向 CMPA:CMPAHR 寄存器执行 32 位写操作时置位；或在向 ↵ CMPA 寄存器执行 16 位写操作时置位。向 CMPAHR 寄存器执行 16 ↵ 位写操作不会影响这个标志。↵ ↵ 一旦发生装载选通，这个位就会自己清零。↵ 0↵ CMPA 映射 FIFO 还未满↵ 1↵ 表示 CMPA 映射 FIFO 满了；CPU 写操作会覆盖当前映射值↵
7↵	Reserved↵	↵	保留↵
6↵	SHDWBMODE↵	↵ 0↵ ↵ 1↵	计数器-比较 B（CMPB）寄存器工作模式↵ 0↵ 映射模式。像一个双缓冲器一样工作。所有通过 CPU 的写操作都可 ↵ 以访问映射寄存器。↵ 1↵ 立即模式。只使用映射比较 B 寄存器。为执行立即比较动作，所有写 操作和读操作都直接访问映射寄存器。↵

4↵	SHDWAMODE↵	↵ 0↵ ↵ 1↵	计数器-比较 A (CMPA) 寄存器工作模式↵ 映射模式。像一个双缓冲器一样工作。所有通过 CPU 的写操作都可以访问映射寄存器。↵ 立即模式。只使用有效比较 A 寄存器。为执行立即比较动作，所有写操作和读操作都直接访问有效寄存器。↵
3-2↵	LOADBMODE↵	↵ ↵ 00↵ 01↵ 10↵ 11↵	有效计数器-比较 B (CMPB) 从映射装载的模式选择位↵ 这个位不影响立即模式 (CMPCTL[SHDWBMODE] = 1) ↵ 在 CTR = 0 时装载：时基计数器等于 0 (TBCTR = 0x0000) ↵ 在 CTR = PRD 时装载：时基计数器等于周期 (TBCTR = TBPRD) 在 CTR = 0 或 CTR = PRD 时装载↵ 停顿（不装载）↵
1-0:↵	LOADAMODE↵	↵ ↵ 00↵ 01↵ 10↵ 11↵	Active 计数器-比较 A (CMPA) 从映射装载的模式选择位↵ 这个位不影响立即模式 (CMPCTL[SHDWAMODE] = 1) ↵ 在 CTR = 0 时装载：时基计数器等于 0 (TBCTR = 0x0000) ↵ 在 CTR = PRD 时装载：时基计数器等于周期 (TBCTR = TBPRD) 在 CTR = 0 或 CTR = PRD 时装载↵ 停顿（不装载）↵

9.2.3 限定器 (AQ) 子模块

动作限定器子模块在构造波形以及产生PWM方面扮演着最重要的角色。它决定将哪些事件转换成哪种动作，从而在EPWMxA和EPWMxB输出上产生所需的开关波形。

动作限定器子模块的用途

动作限定器子模块负责以下事情：

根据以下事件限定并产生动作（**置位、清零、切换**）

CTR = PRD

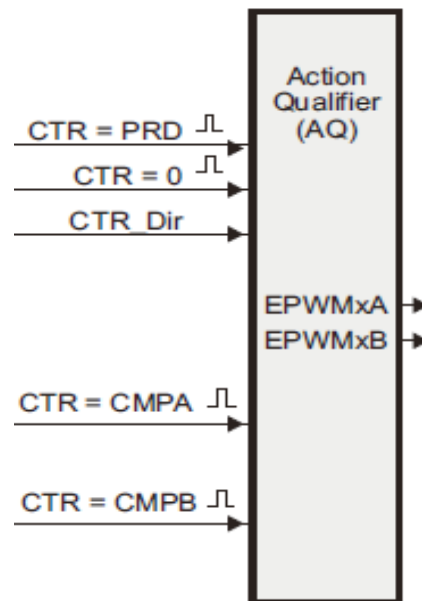
CTR = Zero

CTR = CMPA

CTR = CMPB

在这些事件同时发生时管理优先级

在时基计数器递增和递减时单独控制这些事件



施加到EPWMxA和EPWMxB输出信号上的动作可能是：

置1（高电平）：

将EPWMxA或EPWMxB输出信号设置成高电平

清零（低电平）：

将EPWMxA或EPWMxB输出信号设为低电平

高低电平切换（Toggle）：

如果当前EPWMxA或EPWMxB被拉高，则将输出拉低。

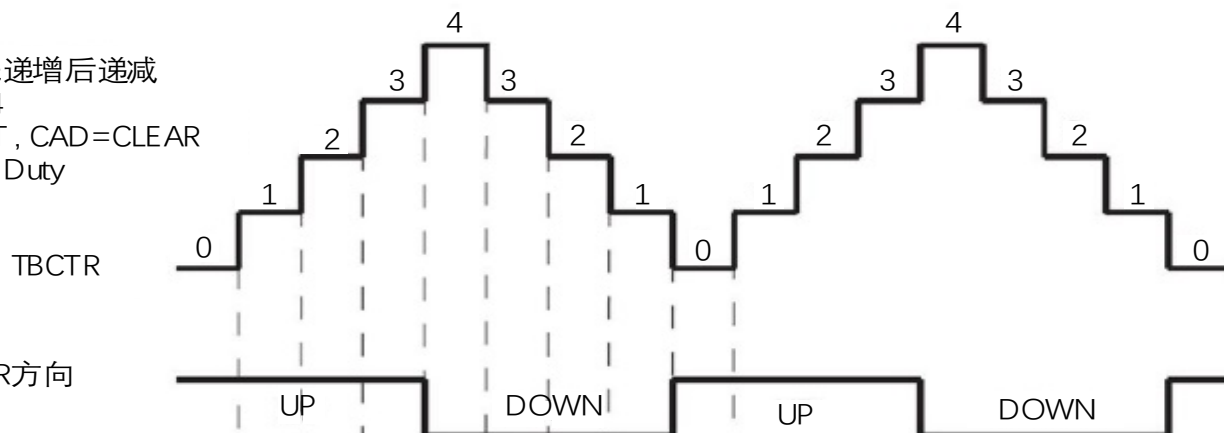
如果当前EPWMxA或EPWMxB被拉低，则将输出拉高。

Do Nothing（不采取任何动作）：

EPWMxA或EPWMxB输出信号保持为当前设置值。尽管

“Do Nothing”选项使得事件不可以在EPWMxA和EPWMxB输出信号上采取动作，但该事件仍然可以触发中断和ADC开始转换。

模式: 先递增后递减
TBPRD=4
CAU=SET, CAD=CLEAR
0%~100% Duty

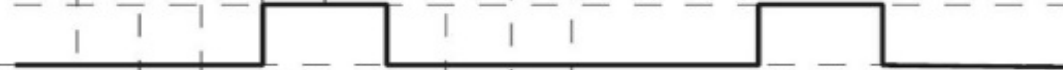


Case 1:
CMPA=4,0%占空比



EPW MxA/EPW MxB

Case 2:
CMPA=3,25%占空比



EPW MxA/EPW MxB

Case 3:
CMPA=2,50%占空比



EPW MxA/EPW MxB

Case 3:
CMPA=1,75%占空比



EPW MxA/EPW MxB

Case 4:
CMPA=0,100%占空比



EPW MxA/EPW MxB

如何输出指定占空比 λ 的PWM?

1. 确定PWM的周期 T_c

2. 确定动作限定器的输出:

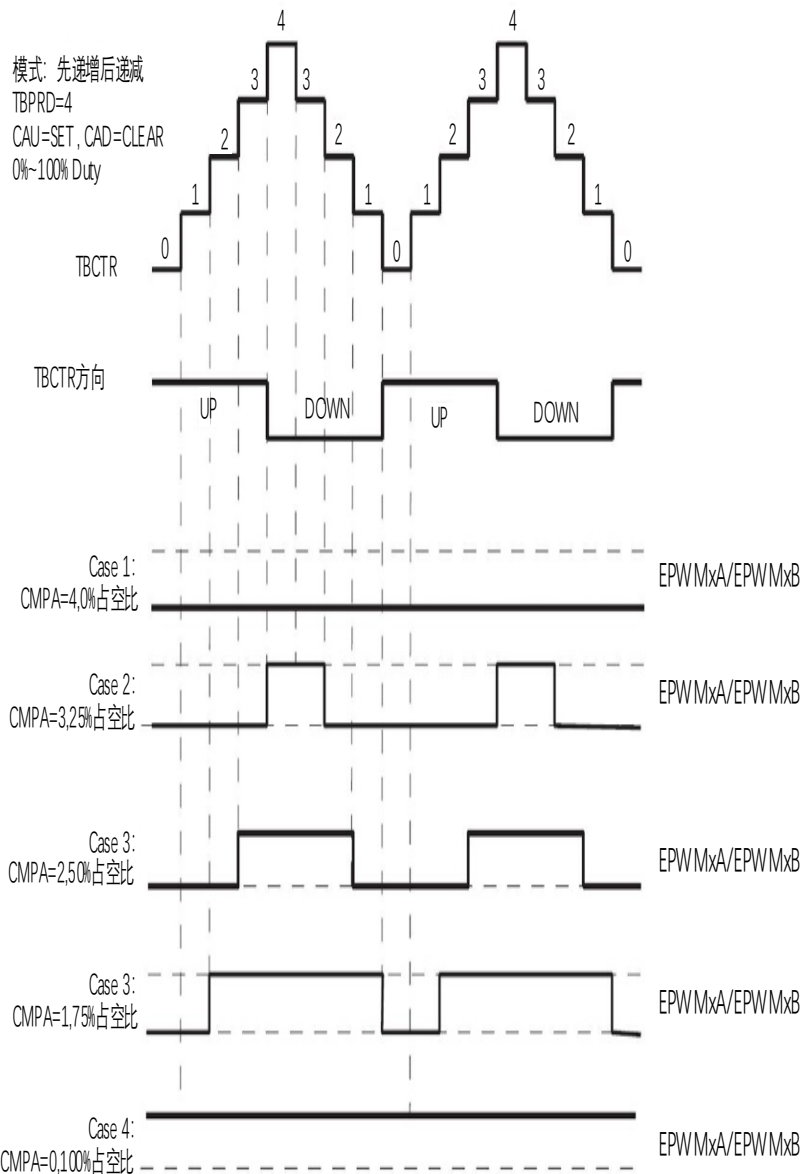
(1) CAU=SET / CAD=CLEAR

(2) CAU=CLEAR / CAD=SET

3. 计算CMPA

$$(1) \text{CMPA} = T_c * (1 - \lambda)$$

$$(2) \text{CMPA} = T_c * \lambda$$



(1) 动作限定器输出A控制寄存器 (AQCTLA)

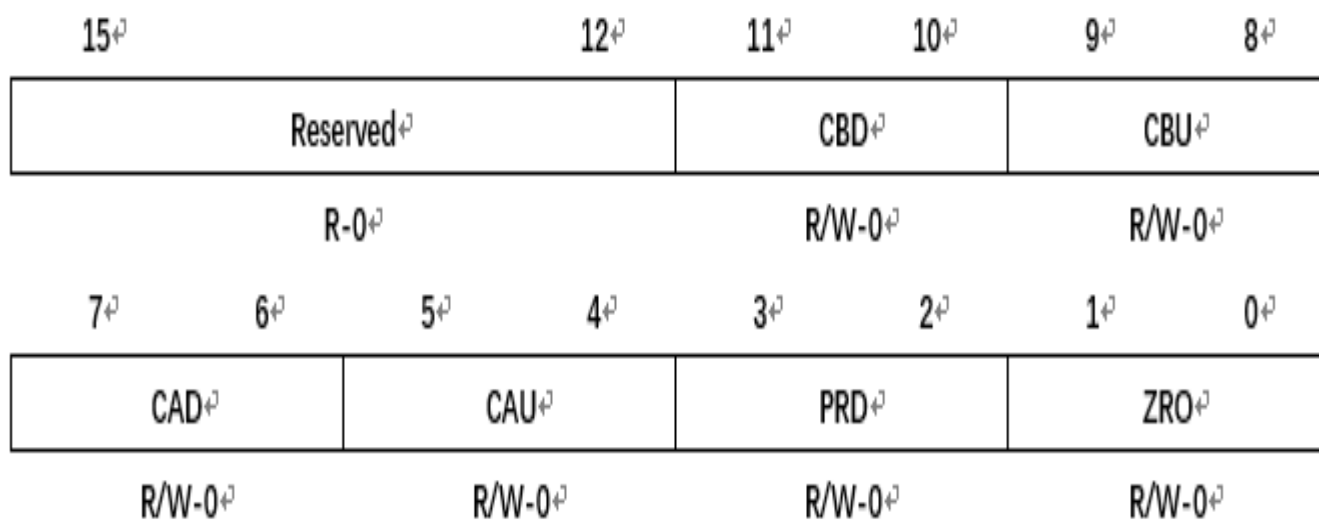


图 9.20 动作限定器输出 A 控制寄存器 (AQCTLA) [↕]

(2) 动作限定器输出B控制寄存器 (AQCTLB)

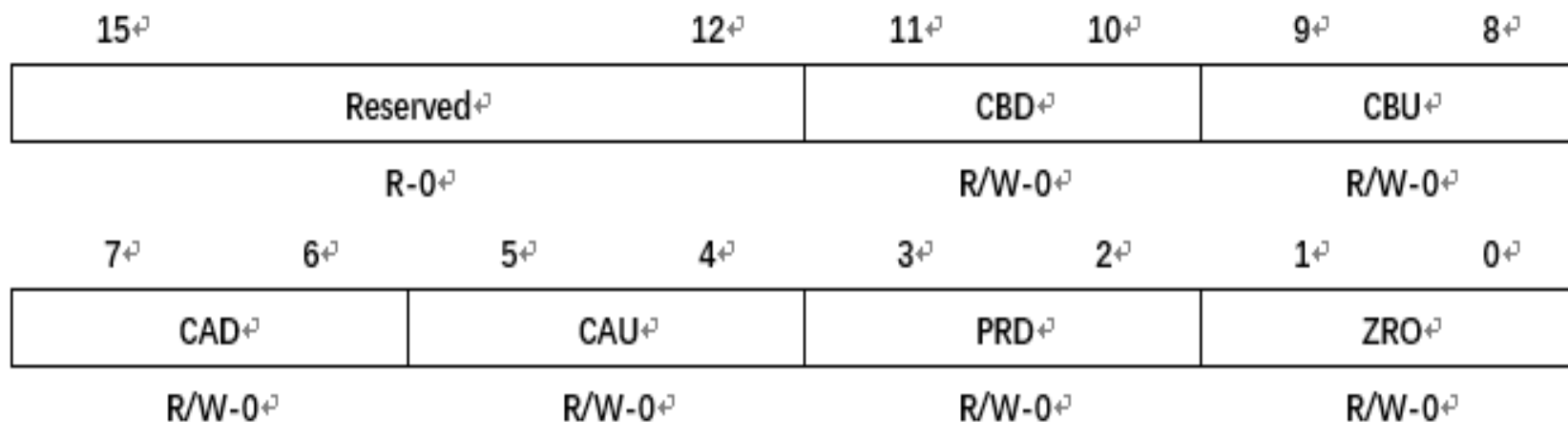


图 9.21 动作限定器输出 B 控制寄存器 (AQCTLB) [↕]

位定义和AQCTLA一样，只是动作的引脚为EPWMxB

(3) 动作限定器软件强制寄存器 (AQSFRC) 和动作限定器连续软件强制寄存器 (AQCSFRC) :

死区模块相关术语

1. AH（Active High）：高有效
2. AL（Active Low）：低有效
3. AHC（Active High Complementary）：互补高有效
4. ALC（Active Low Complementary）：互补低有效
5. RED:上升沿延时
6. FED:下降沿延时

寄存器名称	地址偏移量	是否有shadow寄存器	描述
DBCTL	0x000F	否	死区控制寄存器
DBRED	0x0010	否	死区上升沿延迟计数寄存器
DBFED	0x0011	否	死区下降沿延迟计数寄存器

• 输入源选择：

选择死区模块的输入信号（通过使用DBCTL[IN_MODE]控制位）：

- EPWMxA In是下降沿延迟和上升沿延迟的信号源。这是缺省模式
- EPWMxA In是下降沿延迟的信号源， EPWMxB In是上升沿延迟的信号源
- EPWMxA In是上升沿延迟的信号源， EPWMxB In是下降沿延迟的信号源
- EPWMxB In是下降沿延迟和上升沿延迟的信号源

• 半周期计时：

死区子模块可以用半周期计时方式来计时，以便分辨率翻倍（即计数器以“ $2 \times \text{TBCLK}$ ”计时）

全周期： $\text{FED} = \text{DBFED} \times T_{\text{TBCLK}}$, $\text{RED} = \text{DBRED} \times T_{\text{TBCLK}}$

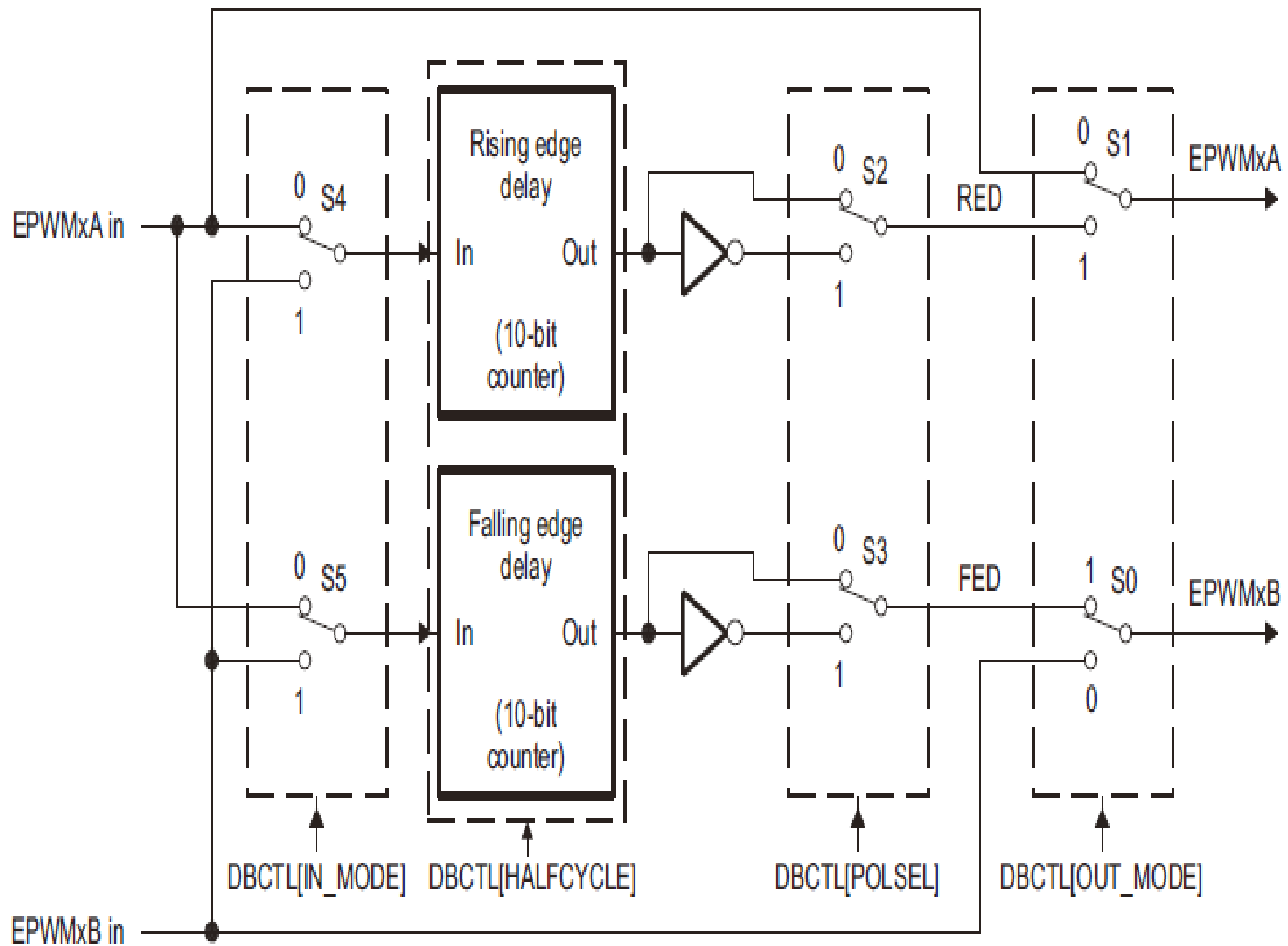
半周期： $\text{FED} = \text{DBFED} \times T_{\text{TBCLK}} / 2$, $\text{RED} = \text{DBRED} \times T_{\text{TBCLK}} / 2$

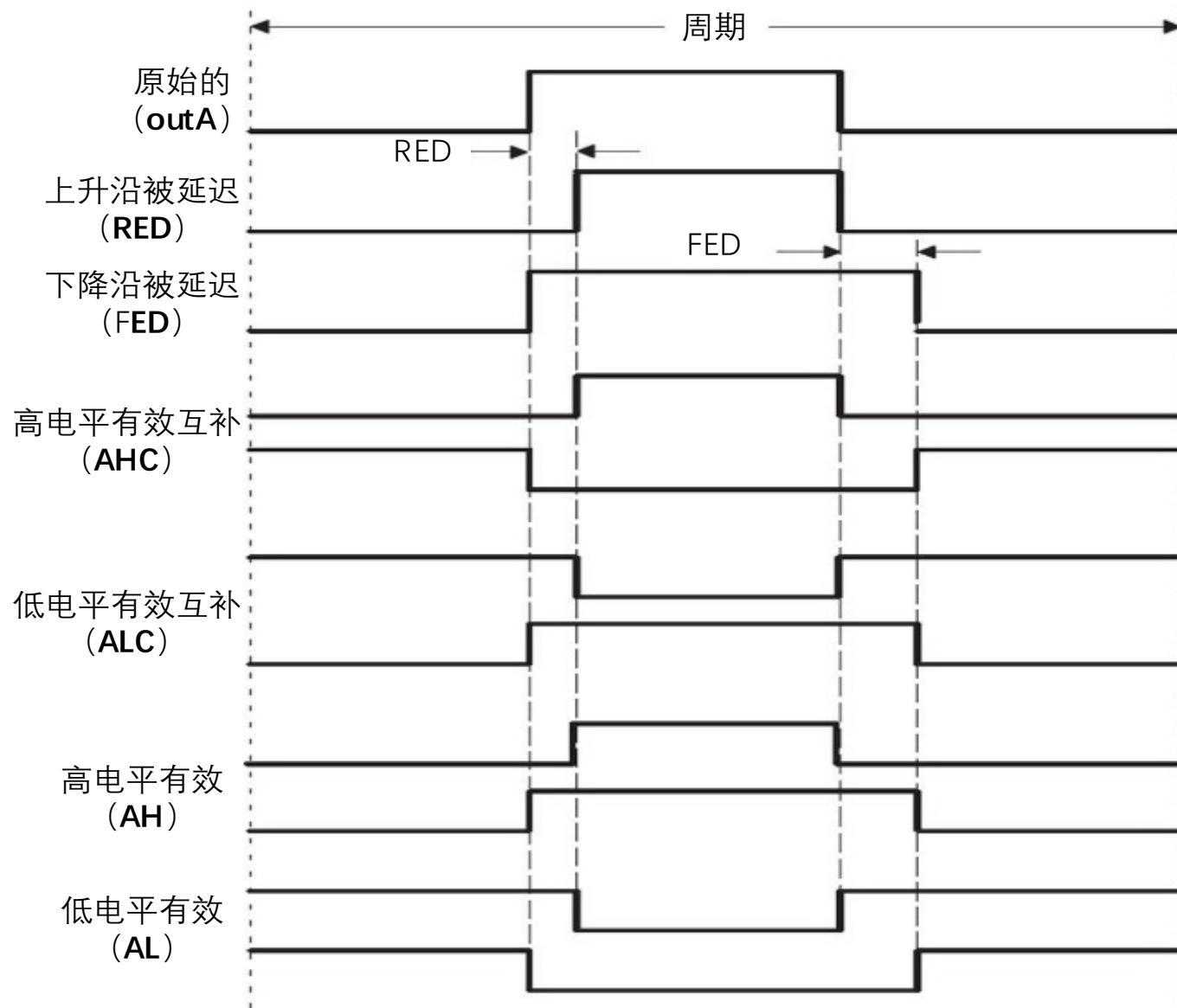
- **输出模式控制：**

输出模式由DBCTL[OUT_MODE]位配置，决定输入信号是添加了下降沿延迟，还是上升沿延迟或者是下降沿与上升沿两种延迟，亦或两种延迟都没有添加。

- **极性控制**

极性控制（DBCTL[POLSEL]）让用户可以确定上升沿延迟信号和/或下降沿延迟信号在从死区子模块发送出去之前是否被反相。





(1) 死区发生器控制寄存器 (DBCTL) :

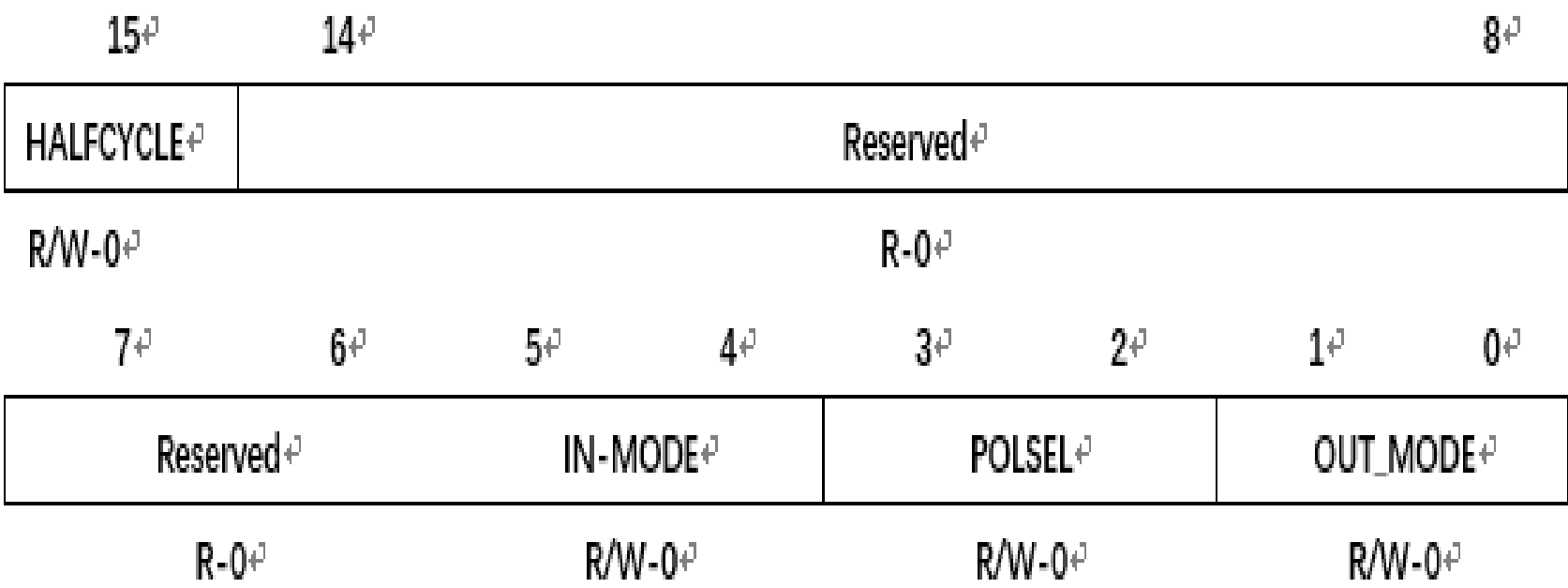


图 9.24 死区发生器控制寄存器 (DBCTL) ↕

表 9.12 死区发生器控制寄存器 (DBCTL) 字段描述

位↵	名称↵	值↵	描述↵
15↵	HALFCYCLE↵	↵ 0↵ 1↵	半周期计时使能位：↵ 全周期计时使能。死区计数器以 TBCLK 速率计时↵ 半周期计时使能。死区计数器以 TBCLK*2 速率计时↵
14-6↵	Reserved↵	↵	保留↵
5-4↵	IN_MODE↵	↵ ↵ ↵ ↵ ↵ 00↵ 01↵ ↵ 10↵ ↵ 11↵	死区输入模式控制↵ 位 5 控制 S5 开关，位 4 控制 S4 开关，如图 9.11 所示。↵ 这个位可用于选择下降沿延迟和上升沿延迟的输入源↵ 要产生传统的死区波形，默认情况下，下降沿延迟和上升沿延迟的源都必须 是 EPWMxA In↵ EPWMxA In（来自动作限定器）是下降沿延迟和上升沿延迟的源↵ EPWMxB In（来自动作限定器）是上升沿延迟信号的源↵ EPWMxA In（来自动作限定器）是下降沿延迟信号的源↵ EPWMxA In（来自动作限定器）是上升沿延迟信号的源↵ EPWMxB In（来自动作限定器）是下降沿延迟信号的源↵ EPWMxB In（来自动作限定器）是上升沿延迟和下降沿延迟信号的源↵

3-2↵	POLSEL↵	↵ 极性选择控制↵ ↵ 位 3 控制 S3 开关，位 2 控制 S2 开关，如图 9.11 所示。↵ ↵ 这个位可用于选择在延迟信号发送到死区子模块之前将它反相↵ ↵ 以下描述与数字电机控制变极器中一条臂（leg）的传统上端/下端开关控制对应。？↵ ↵ 这些位假定 DBCTL[OUT_MODE] = 1,1 且 DBCTL[IN_MODE] = 0,0。其它增强模式也可行，但不是典型模式。↵ 00↵ 高电平有效（AH）模式。EPWMxA 和 EPWMxB 都不反相（缺省）↵ 01↵ 低电平有效互补（ALC）模式。EPWMxA 反相↵ 10↵ 高电平有效互补（AHC）模式。EPWMxB 反相↵ 11↵ 低电平有效（AL）模式。EPWMxA 和 EPWMxB 都反相↵
1-0↵	OUT_MODE↵	↵ 死区输出模式控制↵ ↵ 位 1 控制 S1 开关，位 0 控制 S0 开关，如图 9.11 所示。↵ ↵ 这个位可用于为下降沿延迟和上升沿延迟选择使能或旁路死区发生单元↵ 00↵ 两个输出信号的死区发生单元都被旁路。在该模式下，来自动作限定器的 EPWMxA 和 EPWMxB 输出信号都被直接传输到 PWM 斩波子模块。↵ ↵ 在该模式下，POLSEL 和 IN_MODE 位没什么作用。↵ 01↵ 禁止上升沿延迟。来自动作限定器的 EPWMxA 信号直接传输到 PWM 斩波子模块的 EPWMxA 输入。↵ ↵ 下降沿延迟信号在 EPWMxB 输出上出现。该延迟的输入信号由 DBCTL[IN]MODE]决定。↵ 10↵ 上升沿延迟信号在 EPWMxA 输出上出现。该延迟的输入信号由 DBCTL[IN]MODE]决定。↵ ↵ 禁止下降沿延迟。来自动作限定器的 EPWMxB 信号直接传输到 PWM 斩波子模块的 EPWMxB 输入。↵ 11↵ 针对 EPWMxA 输出上的上升沿延迟以及 EPWMxB 输出上的下降沿延迟，死区完全使能。延迟的输入信号由 DBCTL[IN_MODE]决定。↵

事件触发模块（PWM中断控制）

- 1.接收来自TB模块、CC模块、DC模块事件输入
- 2.可设置每发生1/2/3个事件，产生一个中断或启动ADC
- 3.全可视的寄存器和寄存器控制
- 4.允许软件强制触发中断和启动ADC

(1) 事件触发器选择寄存器 (ETSEL) :

15 ⁺	14 ⁺	⁺	12 ⁺	11 ⁺	10 ⁺	8 ⁺
SOCBEN ⁺	SOCBSEL ⁺			SOCAEN ⁺	SOCASEL ⁺	
R/W-0 ⁺	⁺	R/W-0 ⁺		R/W-0 ⁺	R/W-0 ⁺	
7 ⁺	⁺	4 ⁺	3 ⁺	2 ⁺	0 ⁺	
⁺	Reserved ⁺			INTEN ⁺	INTSEL ⁺	
⁺	R-0 ⁺			R/W-0 ⁺	R/W-0 ⁺	

图 9.27 事件触发器选择寄存器 (ETSEL) ⁺

表 9.13 事件触发器选择寄存器 (ETSEL) 字段描述

位↵	名称↵	值↵	描述↵
15↵	SOCB↵	↵ 0↵ 1↵	使能 ADC 开始转换 B（EPWMxSOCB）脉冲↵ 禁用 EPWMxSOCB↵ 使能 EPWMxSOCB↵
14-12↵	SOCBSEL↵	↵ ↵ 000↵ 001↵ 010↵ 011↵ ↵ 100↵ 101↵ 110↵ 111↵	EPWMxSOCB 选项↵ 这些位决定何时产生 EPWMxSOCB 脉冲。↵ 使能 DCBEVT1.soc 事件↵ 在时基计数器等于 0（TBCTR = 0x0000）时使能事件↵ 在时基计数器等于周期（TBCTR = TBPRD）时使能事件↵ 在时基计数器等于 0 或周期（TBCTR = 0x0000 或 TBCTR = TBPRD） 时使能事件。这种模式在“先递增后递减”计数模式中非常有用↵ 在定时器正在递增时，当时基计数器等于 CMPA 时使能事件↵ 在定时器正在递减时，当时基计数器等于 CMPA 时使能事件↵ 在定时器正在递增时，当时基计数器等于 CMPB 时使能事件↵ 在定时器正在递减时，当时基计数器等于 CMPB 时使能事件↵

11↵	SOCAEN↵	↵	使能 ADC 开始转换 A (EPWMxSOCA) 脉冲↵
		0↵	禁用 EPWMxSOCA↵
		1↵	使能 EPWMxSOCA↵
10-8↵	SOCASEL↵	↵	EPWMxSOCA 选项↵
		↵	这些位决定何时产生 EPWMxSOCA 脉冲。↵
		000↵	使能 DCAEVT1.soc 事件↵
		001↵	在时基计数器等于 0 (TBCTR = 0x0000) 时使能事件↵
		010↵	在时基计数器等于周期 (TBCTR = TBPRD) 时使能事件↵
		011↵	在时基计数器等于 0 或周期 (TBCTR = 0x0000 或 TBCTR = TBPRD)
		↵	时使能事件。这种模式在“先递增后递减”计数模式中非常有用↵
		100↵	在定时器正在递增时，当时基计数器等于 CMPA 时使能事件↵
		101↵	在定时器正在递减时，当时基计数器等于 CMPA 时使能事件↵
		110↵	在定时器正在递增时，当时基计数器等于 CMPB 时使能事件↵
		111↵	在定时器正在递减时，当时基计数器等于 CMPB 时使能事件↵

7-4	Reserved		保留
3	INTEN	0 1	ePWM 中断 (EPWMx_INT) 使能位 禁止产生 EPWMx_INT 中断 允许产生 EPWMx_INT 中断
2-0	INTSEL	000 001 010 011 100 101 110 111	ePWM 中断 (EPWMx_INT) 选项 保留 在时基计数器等于 0 (TBCTR = 0x0000) 时使能事件 在时基计数器等于周期 (TBCTR = TBPRD) 时使能事件 在时基计数器等于 0 或周期 (TBCTR = 0x0000 或 TBCTR = TBPRD) 时使能事件。这种模式在“先递增后递减”计数模式中非常有用 在定时器正在递增时, 当时基计数器等于 CMPA 时使能事件 在定时器正在递减时, 当时基计数器等于 CMPA 时使能事件 在定时器正在递增时, 当时基计数器等于 CMPB 时使能事件 在定时器正在递减时, 当时基计数器等于 CMPB 时使能事件

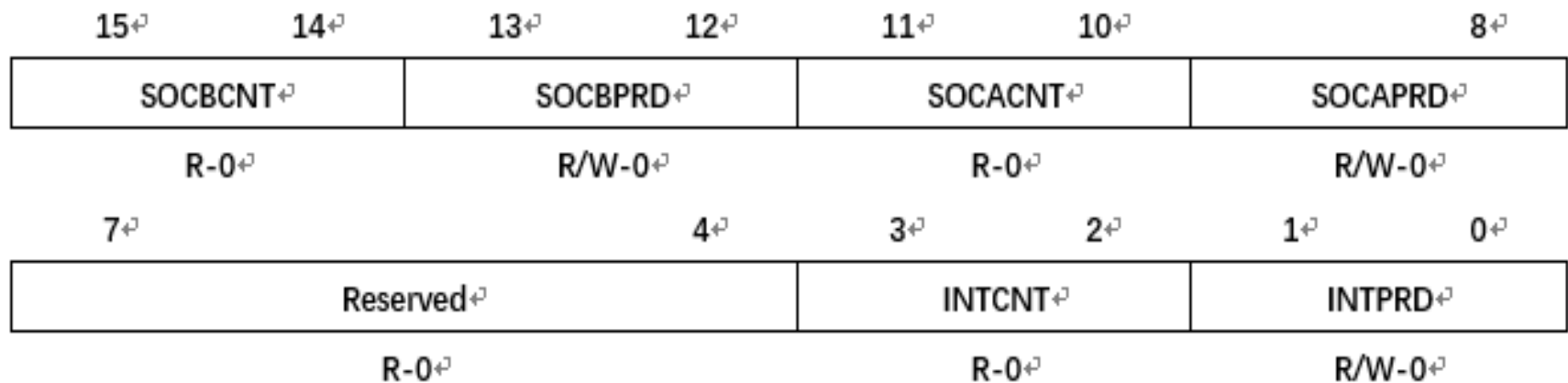


图 9.28 事件触发器预分频寄存器 (ETPS) [↕]

SOCBCNT为ePWM ADC开始转换事件EPWMxSOCB计数值（发生次数）：
0=没有发生事件，1~3表示已经发生了1~3个事件

SOCACNT为ePWM ADC开始转换事件EPWMxSOCA计数值（发生次数）：
0=没有发生事件，1~3表示已经发生了1~3个事件

INTCNT为ePWM中断事件（EPWMx_INT）计数值（发生次数）：
0=没有发生事件，1~3表示已经发生了1~3个事件

ePWM举例:

PWM4A输出一个10Hz直流偏置为1.65V、峰值为3.3V的三角波
(也就是ADCINB7为直流偏置为1.65V、峰值为3.3V的三角波)

设计说明:

1. PWM4A占空比为0时ADCINB7=0V,
PWM4A占空比为50%时ADCINB7=1.65V,
PWM4A占空比为100%时ADCINB7=3.3V。
2. 因此要实现DAC输出,
需要0.05S内占空比从0增加到100%,
再在0.05S内占空比从100%降到0。

3. PWM4可在常用设置基础上作出一些修改（禁止死区或者死区时间为0）即可。

PWM4A的频率为10kHz，每次CNT=0时中断，意味着：

500次PWM中断占空比从0变化到100%（CMPA从0变化到周期值3000），

再500次PWM中断占空比从100%变化到0（CMPA从3000变化到0）。

4.CMPA的变化在PWM4中断服务程序里完成即可，在下面PWM4中断服务程序中PWMTimes为全局变量，用于记录进入PWM4中断的次数。

```
void InitPWM4()
{
    int PWMPRD,DeadTime;
    EALLOW;
    SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 0;
    EDIS;
    PWMPRD = 3000; // 10kHz
    DeadTime = 120; //
    EPwm4Regs.TBPRD = PWMPRD;
    EPwm4Regs.TBPHS.half.TBPHS = 0; //设置相位寄存器为0
    EPwm4Regs.TBCTL.bit.CLKDIV = 0; //CLKDIV = 0;
    EPwm4Regs.TBCTL.bit.HSPCLKDIV = 0; //HSPCLKDIV = 0;
    EPwm4Regs.TBCTL.bit.CTRMODE = 2; //对称模式
    EPwm4Regs.TBCTL.bit.PHSEN = 0; //主模式
    EPwm4Regs.TBCTL.bit.PRDLN = 0;
    EPwm4Regs.TBCTL.bit.SYNCSEL = TB_CTR_ZERO;
    EPwm4Regs.CMPCTL.bit.SHDWAMODE = 0;
    EPwm4Regs.CMPCTL.bit.SHDWBMODE = 0;
```

```
EPwm4Regs.CMPCTL.bit.LOADAMODE = 2; //在CTR=0或者CTR=PRD的时候装载
EPwm4Regs.CMPCTL.bit.LOADBMODE = 2; //同上
EPwm4Regs.AQCTLA.bit.CAU = 1; // EPWM4A在CAU时置1
EPwm4Regs.AQCTLA.bit.CAD = 2;
EPwm4Regs.AQCTLA.bit.CBU = 0;
EPwm4Regs.AQCTLA.bit.CBD = 0;
EPwm4Regs.DBCTL.bit.OUT_MODE = 3; // 使能死区
EPwm4Regs.DBCTL.bit.POLSEL = 2; // 互补高有效
EPwm4Regs.DBFED = DeadTime;
EPwm4Regs.DBRED = DeadTime;
EPwm4Regs.CMPA.half.CMPA = PWMPRD/2;
EPwm4Regs.CMPB = PWMPRD/2;
// Enable CNT_zero interrupt using EPWM4 Time-base
EPwm4Regs.ETSEL.bit.INTEN = 1; //使能EPWM4中断
EPwm4Regs.ETSEL.bit.INTSEL = 1; //CNT_zero时中断
EPwm4Regs.ETPS.bit.INTPRD = 1; //在第一个事件处触发中断
EPwm4Regs.ETCLR.bit.INT = 1; //清中断标志
EALLOW;
SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 1; //同步
EDIS;
}
```

```
interrupt void Epwm4Int_isr(void)
{
    PWMTimes++;
    if(PWMTimes >999)PWMTimes=0;
    if(PWMTimes >500)
        EPwm4Regs.CMPA.half.CMPA=(1000- PWMTimes)*6;
    else EPwm4Regs.CMPA.half.CMPA=PWMTimes * 6;

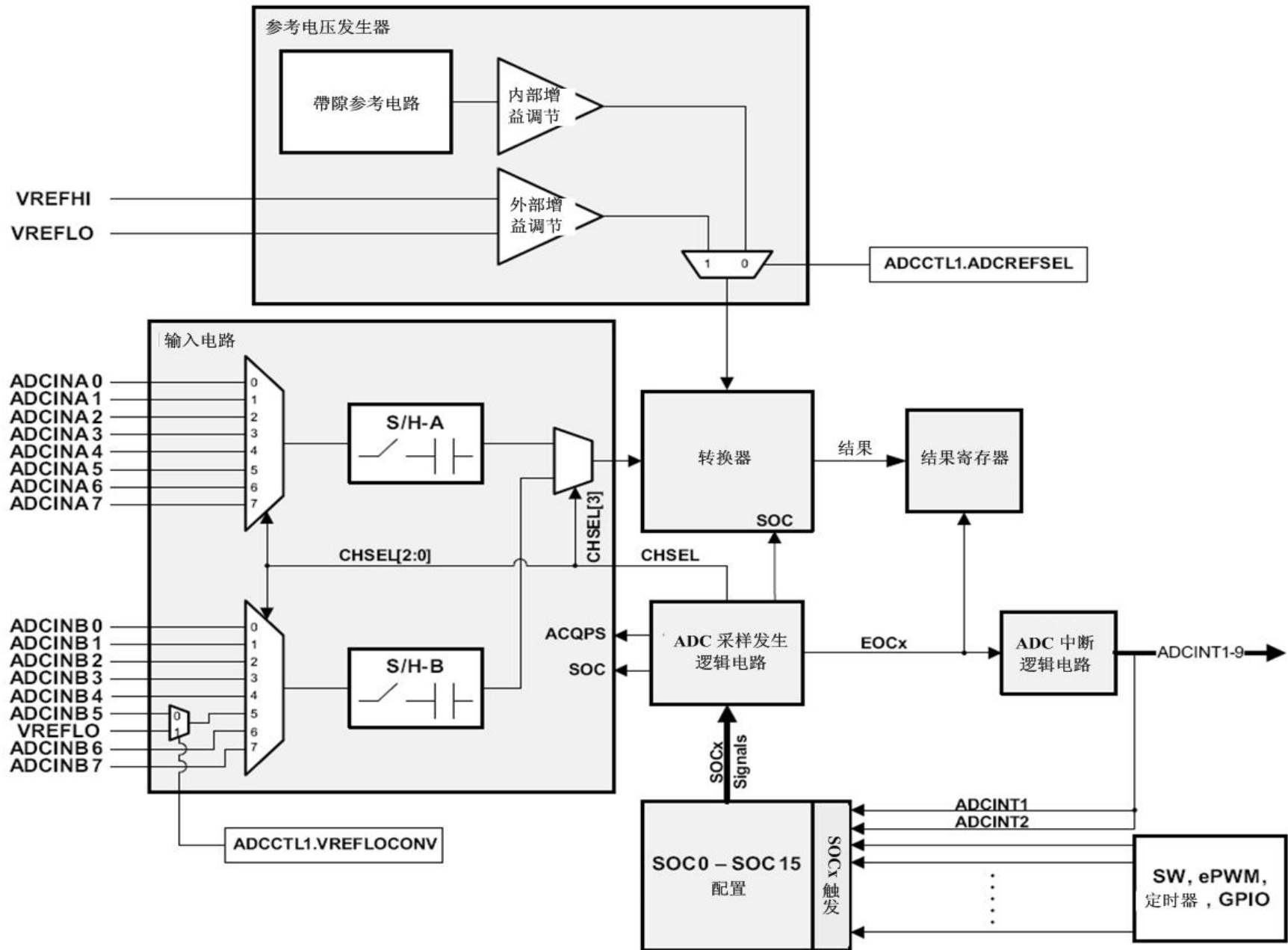
    EPwm4Regs.ETCLR.bit.INT = 1;
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP3;
    // EINT;
}
```


1) 这种DAC的分辨率取决于PWM的周期，PWM周期越长，分辨率越高。

2) 这种DAC输出的模拟信号的频率受制于低通滤波器的特性、PWM的频率和模拟信号的分辨率。

这种DAC往往用在低成本、低频场合。

10.1 F2802X的ADC内核介绍和寄存器列表



10.2 SOC的工作原理

- ✓ 与之前的那些ADC不同，这个ADC不基于序列发生器(sequencer)，而是基于SOC
- ✓ 术语SOC是一种配置设置，它定义的是单通道单转换
- ✓ 该设置包含3个配置：启动转换的触发源、转换通道和采集(采样)窗口尺寸
- ✓ 每个SOC都是单独配置的，并且可以是触发源、通道和/或采集窗口的任意组合
- ✓ 这一特性提供了一种非常灵活的转换配置方式，包括从“使用不同触发器、不同通道的单独采样”到“使用单个触发器、相同通道的过采样”

- ✓ SOCx的触发源由ADCSOCxCTL寄存器中的TRIGSEL字段以及ADCINTSOCSEL1或ADCINTSOCSEL2寄存器中的相应位联合配置
- ✓ 软件通过ADCSOCFRC1寄存器也可以强制产生一个SOC事件
- ✓ SOCx的通道和采样窗尺寸都通过ADCSOCxCTL寄存器的CHSEL和ACQPS字段来配置

10.2.1 ADC采集(采样和保持)窗口

- ✓ 各外部驱动器驱动模拟信号的能力各不相同，有些电路需要更长的时间才能正确地将电荷(charge)传送到ADC的采样电容器(sampling capacitor)
- ✓ 为解决这个问题，**可对每个SOC采样窗口的长度进行控制，每个ADC SOCxCTL寄存器都包含一个6位字段ACQPS，该字段决定采样和保持(S/H)窗的长度**
- ✓ 对于SOC来说，写入该字段的值要比采样窗的期望值(单位为周期数)小1，如果该字段为15,则表示采样时间其实为16个时钟周期,采样周期的最小值为7，(ACQPS=6)
- ✓ **总采样时间等于采样窗长度加上ADC转换时间(共13个ADC时钟)**

ADC时钟	ACQPS	采样窗口	转换时间 (13个周期)	处理模拟 电压的总时间
60MHz	6	116.67ns	216.67ns	333.34ns
60MHz	8	150.00ns	216.67ns	366.67ns
60MHz	10	183.33ns	216.67ns	400.00ns
60MHz	14	250.00ns	216.67ns	466.67ns
60MHz	25	433.33ns	216.67ns	650.00ns
40MHz	6	175ns	325ns	500.00ns
40MHz	25	650ns	325ns	975.00ns

10.2.2 触发操作

- ✓ 每个SOC都可以配置成由许多输入触发器中的其中一个启动。如果需要，几个SOC可以配置成使用**同一个通道**
- ✓ 一些可用的输入触发器：
 - 软件
 - CPU定时器0/1/2
 - XINT2 SOC
 - ePWM1 ~ 7 SOCA和SOCB
- ✓ ADCINT1和ADCINT2可以反馈回来触发另外一次转换，这种配置由ADCINTSOCSEL1/2寄存器控制，**如果需要连续转换，这种模式将非常有用**

10.2.3 通道选择

- ✓ 每个SOC都可以配置成转换任意一个可用的ADCIN输入通道
- ✓ 当SOC被配置成顺序采样模式时，由ADC SOCxCTL寄存器的4位CHSEL字段决定要转换的通道
- ✓ 当SOC被配置成同步采样模式时，CHSEL字段的最高有效位(第4位)将被丢弃且由低3位决定转换哪一对通道
- ✓ ADCINA0与VREFHI复用，因此在使用外部电压基准时，它不能用作一个可变的输入源

ADC SOC0 ~ SOC15控制寄存器 (ADCSOCxCTL)

15	11	10	9	6	5	0
TRIGSEL		Reserved	CHSEL		ACQPS	
R/W-0		R-0	R/W-0		R/W-0	

位	名称	值	描述
15-11	TRIGSEL	00h 01h 02h 03h 04h 05h 06h 07h 08h 09h 0Ah 0Bh 0Ch 0Dh 0Eh 0Fh 10h 11h 12h 其它值	SOCx触发源选择位。 配置由哪个触发源在ADCSOCFLG1寄存器中设置SOCx标志并在SOCx拥有优先权时启动转换。ADCINTSOCSEL1或ADCINTSOCSEL2寄存器中相应的SOCx字段的值优先于TRIGSEL字段。 ADCTRIG0——仅由软件触发 ADCTRIG1——CPU定时器0.TINT0n ADCTRIG2——CPU定时器1.TINT1n ADCTRIG3——CPU定时器2.TINT2n ADCTRIG4——XINT2.XINT2SOC ADCTRIG5——ePWM1.ADCSOCA ADCTRIG6——ePWM1.ADCSOCB ADCTRIG7——ePWM2.ADCSOCA ADCTRIG8——ePWM2.ADCSOCB ADCTRIG9——ePWM3.ADCSOCA ADCTRIG10——ePWM3.ADCSOCB ADCTRIG11——ePWM4.ADCSOCA ADCTRIG12——ePWM4.ADCSOCB ADCTRIG13——ePWM5.ADCSOCA ADCTRIG14——ePWM5.ADCSOCB ADCTRIG15——ePWM6.ADCSOCA ADCTRIG16——ePWM6.ADCSOCB ADCTRIG17——ePWM7.ADCSOCA ADCTRIG18——ePWM7.ADCSOCB 无效选择
10	Reserved		读返回0；对该位执行写操作无影响

位	名称	值	描述
9-6	CHSEL		SOCx通道选择位。用于在ADC接收到SOCx时选择将被转换的通道。
			顺序采样模式（SIMULEN _x = 0）：
		0h	ADCINA0
		1h	ADCINA1
		2h	ADCINA2
		3h	ADCINA3
		4h	ADCINA4
		5h	ADCINA5
		6h	ADCINA6
		7h	ADCINA7
		8h	ADCINB0
		9h	ADCINB1
		Ah	ADCINB2
		Bh	ADCINB3
		Ch	ADCINB4
		Dh	ADCINB5
		Eh	ADCINB6
		Fh	ADCINB7
			同步采样模式（SIMULEN _x = 1）：
		0h	ADCINA0/ADCINB0对
		1h	ADCINA1/ADCINB1对
		2h	ADCINA2/ADCINB2对
		3h	ADCINA3/ADCINB3对
		4h	ADCINA4/ADCINB4对
		5h	ADCINA5/ADCINB5对
		6h	ADCINA6/ADCINB6对
		7h	ADCINA7/ADCINB7对
		8h	无效选择
		9h	无效选择
		Ah	无效选择
		Bh	无效选择
		Ch	无效选择
		Dh	无效选择
		Eh	无效选择
		Fh	无效选择

位	名称	值	描述
5-0	ACQPS	00h 01h 02h 03h 04h 05h 06h 07h 08h 09h ... 3Fh	SOCx采集预分频控制位。该位用于控制SOCx其采样/保持窗口的长度，最小值可以为6。 无效选择 无效选择 无效选择 无效选择 无效选择 无效选择 采样窗口7个周期长（6+1个时钟周期） 采样窗口8个周期长（7+1个时钟周期） 采样窗口9个周期长（8+1个时钟周期） 采样窗口10个周期长（9+1个时钟周期） 采样窗口64个周期长（63+1个时钟周期）

10.2.4 SOC配置其他注意事项

每个SOC都是单独配置的，
并且可以是触发源、通道和/或采集窗口的任意组合。

这一特性提供了一种非常灵活的转换配置方式：
包括从“使用不同触发器、不同通道的单独采样”
到“使用单个触发器、相同通道的过采样”。

在ADC的应用实例中还必须通过**PCLKCR0**寄存器将时钟使能，且**必须让ADC上电并正常工作**。

举例

- ✓ 如果要将通道ADCINA1上的单一转换设置成在ePWM3定时器到达周期值时发生
- ✓ 首先必须将ePWM3设置成在到达周期值时输出一个SOCA或SOCB信号。此时，我们选择SOCA。
- ✓ 然后将其中一个SOC设置成使用其ADC SOCxCTL寄存器。由于选择哪个SOC信号都没有什么区别，所以我们使用SOC0。
- ✓ ADC允许采样窗最短为7个周期。如果要让采样窗的时间最短，转换通道为ADCINA1，SOC0触发器为ePWM3，我们必须分别将ACQPS字段设为6，CHSEL字段设为1，TRIGSEL字段设为9。这样，写入寄存器的结果值将为：

15	11	10	9	6	5	0
TRIGSEL		Reserved	CHSEL		ACQPS	
R/W-0		R-0	R/W-0		R/W-0	
01001		0	0001		000110	

ADC SOC0CTL = 4846h;

- ✓ 如果配置成这样的话，ePWM3 SOCA事件将启动一次ADCINA1转换，并将结果值存放在ADCRESULT0寄存器中

15	11	10	9	6	5	0
TRIGSEL		Reserved		CHSEL		ACQPS
R/W-0		R-0		R/W-0		R/W-0

- ✓ 如果ADCINA1需要被3次**过采样**，那么可以将SOC1、SOC2和SOC3配置成和SOC0一样

ADCSOC1CTL = 4846h;

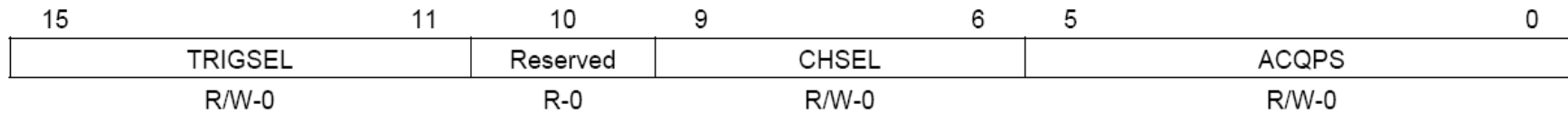
ADCSOC2CTL = 4846h;

ADCSOC3CTL = 4846h;

- ✓ 如果配置成这样的话，ePWM3 SOCA事件将连续启动4次ADCINA1转换，并将结果值分别存放在ADCRESULT0 ~ ADCRESULT3寄存器中

过采样

- 可以提高ADC的精度
- 合理编排，可以起到其他方面很好的效果，比如积分、滤波等等
- 在DSP控制中广泛采样



- ✓ 可能需要从同一触发器采样3个不同的信号。这可以通过改变SOC0 ~ SOC2的CHSEL字段并将TRIGSEL字段保持不变来实现

ADCSOC0CTL = 4846h; // (ACQPS = 6, CHSEL = 1, TRIGSEL = 9)

ADCSOC1CTL = 4886h; // (ACQPS = 6, CHSEL = 2, TRIGSEL = 9)

ADCSOC2CTL = 48C6h; // (ACQPS = 6, CHSEL = 3, TRIGSEL = 9)

- ✓ 如果配置成这样的话，ePWM3 SOCA事件将连续启动3次转换。通道ADCINA1上转换的结果将在ADCRESULT0中显示。ADCINA2通道上转换的结果将在ADCRESULT1中显示。ADCINA3通道上转换的结果将在ADCRESULT2中显示。
- ✓ 转换通道和触发器这两者与转换结果在何处显示没有任何关系。RESULT寄存器与SOC有关

(2) 同步采样模式

在某些应用中需要保证两个信号之间的采样延迟最短。ADC包含两个采样/保持电路，允许同时对两个不同的通道进行采样。

同步采样模式使用ADCSAMPLEMODE寄存器对一对SOCx进行配置。

偶数编号的SOCx和它之后的奇数编号的SOCx(即SOC0和SOC1)配成一对，一起连接一个使能位(此时为SIMULEN0)

配对规则如下：

- ✓ 任意一个SOCx触发源都可以启动一对转换
- ✓ 那对转换通道将由A通道和B通道组成，这两个通道由被触发SOCx的CHSEL值决定。这种模式下，有效值为0 ~ 7
- ✓ 同时采样两个通道
- ✓ 一直是首先转换A通道
- ✓ A通道转换后将会产生偶数编号的EOCx脉冲，B通道转换后则产生奇数编号的EOCx脉冲
- ✓ A通道的转换结果存放在偶数编号的ADCRESULTx寄存器中，B通道的转换结果则存放在奇数编号的ADCRESULTx寄存器中

举例

如果ADCSAMPLEMODE.SIMULEN0=1且SOC0配置为

CHSEL = 2(ADCINA2/ADCINB2通道);

TRIGSEL = 5(ADCTRIG5=ePWM1.ADCSOCA);

当ePWM1发出一个ADCSOCA触发事件时，ADCINA2和ADCINB2将会同时被采样。

之后ADCINA2通道将会立即被转换，且转换结果存放在ADCRESULT0寄存器中。

根据ADCCTL.INTPULSEPOS的设置，在ADCINA2转换开始或结束时将出现EOC0脉冲。接着ADCINB2通道被转换，且转换结果存放在ADCRESULT1寄存器中。

根据ADCCTL1.INTPULSEPOS的设置，在ADCINB2转换开始或结束时将出现EOC1脉冲

(3) EOC和中断操作

就像有16组独立的SOCx配置一样，同样也有16个EOCx脉冲。

在顺序采样模式下，EOCx与SOCx是直接相关的。

在同步采样模式下，“偶数EOCx和其后的奇数EOCx”与“偶数SOCx和其后的奇数SOCx”是相关的，如上节所述。根据ADCCTL1.INTPULSEPOS的设置，EOCx脉冲将在转换开始或转换结束时出现

ADC包含9个中断，这些中断可以被标记(flag)和/或传送到PIE。

其中每一个中断都可以配置成接收任何一个EOCx信号作为其触发源。

至于哪个EOCx是触发源，则由INTSELxNy寄存器决定。

此外，ADCINT1和ADCINT2信号可以配置成产生一个SOCx触发事件。这对连续转换来说非常有用

(6) 参考电压的选择

内部参考电压

默认情况下选择内部带隙(bandgap)来产生ADC参考电压。这样一来，将根据固定的0 ~ 3.3V量程范围来转换电压。控制这种转换模式的等式如下

Digital Value = 0 输入 $\leq$ 0v时

Digital Value = $4096 * [(Input - VREFLO) / 3.3v]$ 0v < 输入 < 3.3v时

Digital Value = 4095, 输入 $\geq$ 3.3v时

注：

所有小数部分都被截掉
在该模式下，VREFLO必须与地连接

外部参考电压

若要将一个输入电压按比例转换，则应该选择外部VREFH/VREFLO管脚作为参考电压的引脚。与内部带隙模式的0 ~ 3.3v固定输入范围不同，比例模式的输入范围是VREFLO到VREFHI。转换值将被调节到该范围内。例如，如果VREFLO被设置成0.5v且VREFHI被设置成3.0v，那么1.75v电压将被转换成数字2048。VREFLO和VREFHI的允许范围请参阅器件具体的数据手册。在一些器件中，VREFLO与地在内部连接，因此被限制为0V。控制这种转换模式的等式如下：

Digital Value = 0 输入 $\leq$ VREFLO时

Digital Value = $4096 * [(Input - VREFLO) / (VREFHI - VREFLO)]$
VREFLO < 输入 < VREFHI时

Digital Value = 4095 输入 $\geq$ VREFHI时

1) ADC初始化程序

```
void InitADC()
```

```
{
```

```
    int DCSampT;
```

```
    DCSampT=15;
```

```
    EALLOW;
```

```
    SysCtrlRegs.PCLKCR0.bit.ADCENCLK = 1;
```

```
    (*Device_cal)();
```

```
    EDIS;
```

```
    DELAY_US(ADC_usDELAY);
```

```
    EALLOW;
```

```
    AdcRegs.ADCCTL1.bit.ADCBGPWD = 1;// 内核内的带隙缓冲器电路上电
```

```
    AdcRegs.ADCCTL1.bit.ADCREFPWD = 1;//内核内的参考缓冲器电路上电
```

```
    AdcRegs.ADCCTL1.bit.ADCPWDN = 1;// 内核内的模拟电路上电
```

```
    AdcRegs.ADCCTL1.bit.ADCENABLE = 1; // ADC使能
```

```
    AdcRegs.ADCCTL1.bit.ADCREFSEL = 1;
```

```
                                // 使用外部VREFHI/VREFLO参考电压
```

```
    //AdcRegs.ADCCTL1.bit.ADCREFSEL = 0;// 使用内部带隙产生参考电压
```

```
    EDIS;
```

```
DELAY_US(ADC_usDELAY); // 转换ADC通道前延迟
AdcOffsetSelfCal();
DELAY_US(ADC_usDELAY); // 转换ADC通道前延迟
EALLOW;
AdcRegs.ADCCTL1.bit.INTPULSEPOS= 1;
                                //转换完成前一个ADC时钟周期产生EOC
AdcRegs.INTSEL1N2.bit.INT1E    = 1;  // ADCINT1使能
AdcRegs.INTSEL1N2.bit.INT1CONT = 0; //禁用ADCINT 1连续模式
AdcRegs.INTSEL1N2.bit.INT1SEL= 0x0f;
                                //设置EOC 15以触发ADCINT 1启动
AdcRegs.ADCSAMPLEMODE.all = 0xff; //SOCAx和SOCBx同步采样
//对ADCINA7、ADCINB7过采样
AdcRegs.ADCSOC0CTL.bit.CHSEL = 0x07; //A7,Result0; B7,Result1
AdcRegs.ADCSOC2CTL.bit.CHSEL = 0x07; //A7,Result2; B7,Result3
AdcRegs.ADCSOC4CTL.bit.CHSEL = 0x07; //A7,Result4; B7,Result5
AdcRegs.ADCSOC6CTL.bit.CHSEL = 0x07; //A7,Result6; B7,Result7
//对ADCINA3、ADCINB3过采样
```



```
AdcRegs.ADCSOC8CTL.bit.CHSEL = 0x03; //A3,Result8; B3,Result9
AdcRegs.ADCSOC10CTL.bit.CHSEL = 0x03; //A3,Result10; B3Result11
AdcRegs.ADCSOC12CTL.bit.CHSEL = 0x03; //A3,Result12; B3,Result13
AdcRegs.ADCSOC14CTL.bit.CHSEL = 0x03; //A3,Result14; B3,Result15
AdcRegs.ADCSOC0CTL.bit.TRIGSEL= 5;
        //ADCTRIG5 – ePWM1, ADCSOCA
AdcRegs.ADCSOC2CTL.bit.TRIGSEL= 5;
AdcRegs.ADCSOC4CTL.bit.TRIGSEL= 5;
AdcRegs.ADCSOC6CTL.bit.TRIGSEL= 5;
AdcRegs.ADCSOC8CTL.bit.TRIGSEL= 5;
AdcRegs.ADCSOC10CTL.bit.TRIGSEL= 5;
AdcRegs.ADCSOC12CTL.bit.TRIGSEL= 5;
AdcRegs.ADCSOC14CTL.bit.TRIGSEL= 5;
```

```
AdcRegs.ADCSOC0CTL.bit.ACQPS= ADCSampT;  
                                //设置SOC0~1采样窗口的长度
```

```
AdcRegs.ADCSOC2CTL.bit.ACQPS= ADCSampT;
```

```
AdcRegs.ADCSOC4CTL.bit.ACQPS= ADCSampT;
```

```
AdcRegs.ADCSOC6CTL.bit.ACQPS= ADCSampT;
```

```
AdcRegs.ADCSOC8CTL.bit.ACQPS= ADCSampT;
```

```
AdcRegs.ADCSOC10CTL.bit.ACQPS = ADCSampT;
```

```
AdcRegs.ADCSOC12CTL.bit.ACQPS = ADCSampT;
```

```
AdcRegs.ADCSOC14CTL.bit.ACQPS = ADCSampT;
```

```
EDIS;
```

```
EPwm1Regs.ETSEL.bit.SOCAEN= 1;// 使能EPWM1SOCA
```

```
EPwm1Regs.ETSEL.bit.SOCASEL= 1;//在CTR = ZERO时使能事件
```

```
EPwm1Regs.ETPS.bit.SOCAPRD = 1;// 在第一个事件产生脉冲
```

```
EPwm1Regs.ETCLR.bit.SOCA = 1;    // 清除SOCA标志
```

```
}
```

2) ADC中断服务程序

```
interrupt void ADC1Int_isr(void)
```

```
{
```

```
    ADCA7 = AdcRegs.ADCRESULT0+AdcRegs.ADCRESULT2;
```

```
    ADCA7 += AdcRegs.ADCRESULT4+AdcRegs.ADCRESULT6;
```

```
    ADCA3 = AdcRegs.ADCRESULT6+AdcRegs.ADCRESULT10;
```

```
    ADCA3 += AdcRegs.ADCRESULT12+AdcRegs.ADCRESULT14;
```

```
    ADCB7 = AdcRegs.ADCRESULT1+AdcRegs.ADCRESULT3;
```

```
    ADCB7 += AdcRegs.ADCRESULT5+AdcRegs.ADCRESULT7;
```

```
    ADCB3 = AdcRegs.ADCRESULT9+AdcRegs.ADCRESULT11;
```

```
    ADCB3 += AdcRegs.ADCRESULT13+AdcRegs.ADCRESULT15;
```

```
    AdcRegs.ADCINTFLGCLR.bit.ADCINT1 = 1;
```

```
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
```

```
    EINT;
```

```
}
```

程序中，ADCA7、ADCA3、ADCB7和ADCB3分别对应于通道ADCINA7、ADCINA3、ADCINB7和ADCINB3转换的结果。

ADC应用举例:小车供电电池电压测量：小车采用两颗12V电池供电，两路电池电压通过电阻分压（分压比为3:1，也就是电池电压除以4）后引入到F28027的ADCINA3和ADCINB3。
要求：填空完成ADC的初始化程序和中断服务程序。

```
void InitADC()
{
    .....

    AdcRegs.INTSEL1N2.bit. (1) = 1; // ADCINT1使能
    AdcRegs.INTSEL1N2.bit.INT1CONT = 0;
    AdcRegs.INTSEL1N2.bit.INT1SEL = (2);
        //EOC 3以触发ADCINT1
    AdcRegs.ADCSAMPLEMODE.all = (3); //同步采样
    AdcRegs.ADCSOC0CTL.all = (4);
        // ePWM1的ADCSOCA触发，ACQPS=6
    AdcRegs.ADCSOC2CTL.all = (4);
        // ePWM1的ADCSOCA触发，ACQPS=6
    EPwm1Regs.ETSEL.bit. (5) = 1; // 使能 EPWM1_SOCA
    EPwm1Regs.ETSEL.bit.SOCASEL = (6);
        //在CTR = PRD时使能事件
    EPwm1Regs.ETPS.bit.SOCAPRD = (7);
        // 在第一个事件产生脉冲
    EPwm1Regs.ETCLR.bit.SOCA = 1; // 清除 SOCA标志
    .....
}
```

```
int ADCA3, ADCB3;
float VolCell1, VolCell2; //电池电压

interrupt void ADC1Int_isr(void)
{
    ADCA3 = AdcRegs.ADCRESULT0;
    ADCA3 += AdcRegs.ADCRESULT2;
    ADCB3 = AdcRegs.ADCRESULT1;
    ADCB3 += AdcRegs.ADCRESULT3;
    VolCell1 = ADCA3 * (8);
        //(8)为折算系数
    VolCell2 = ADCB3 * (8);
    AdcRegs.ADCINTFLGCLR.bit. (9) = 1;
    PieCtrlRegs.PIEACK.all = (10);
}
```